圆柱形中药材热风烘干过程的二维热质耦合传递建模与计算方法

摘　要

本文基于传热传质理论, 同时考虑径向和轴向的传递过程, 建立了二维轴对称热质传递数学模型, 采用有限体积法进行数值求解.

对于问题一, 本文从傅里叶导热定律出发, 在常物性参数假设下建立了圆柱坐标系下的预热平衡阶段传热传质模型. 边界条件考虑表面对流换热、对流传质及水分蒸发吸热, 中心采用对称条件. 空间离散采用有限体积法对药材横截面进行网格剖分, 时间积分采用隐式格式以保证数值稳定性, 逐时间步求解药材内部的温度场与水分浓度场的演化过程, 获得了预热平衡阶段温度与水分的基本分布特征.

对于问题二, 本文将密度、比热容、热传导系数推广为含水率的函数, 水分扩散系数同时依赖含水率与温度, 使传热方程与传质方程通过物性参数形成双向耦合. 求解策略上采用交替迭代法, 每个时间步长内先以当前含水率场更新热物性参数求解温度场, 再以更新后的温度场更新扩散系数求解含水率场, 反复迭代直至两场收敛后推进至下一时间步, 实现了对预热平衡与恒温干燥两个完整阶段的连续仿真.

对于问题三, 在问题二的耦合模型基础上, 本题以预热平衡与恒温干燥为条件, 以全域最大含水率低于给定阈值作为烘干合格判据, 通过持续时间推进仿真, 追踪各空间节点含水率随时间的变化轨迹. 据此确定了恒温干燥阶段所需的总时长, 为烘干工艺的时间规划提供了定量依据.

对于问题四, 本文引入了药材因水分流失而产生的几何收缩效应, 通过坐标变换将随时间变化的移动边界映射至固定的计算域. 变换后的控制方程中引入了由收缩速率引起的附加对流项, 使得原方程在固定网格上即可求解含移动边界的传热传质问题, 进而分析了收缩对内部温度与水分分布的影响规律.

关键词: 偏微分方程运动边界问题, 抛物型方程初边值问题, 扩散与对流传热传质, 有限体积法.

MSC codes: 35R37, 35K20, 80A19, 80M12.

一、 问题重述

中药材热风烘干是药材加工中十分关键的工艺, 不合理的工艺参数会造成干燥效率低、能耗高、药材成品品质不稳定等问题. 传统试验优化手段成本高、周期长. 因此需要借助数学建模与数值仿真的方法, 探究圆柱形中药材烘干过程内部温度与水分的动态演化规律.

问题一: 某圆柱形中药材长 25 cm、半径 2 cm, 干燥初始时刻药材温度为 28 °C, 水分浓度 (干基含水率) 为 $2.55\,\mathrm{kg\,kg^{-1}}$, 烘房温度与水分浓度随时间的变化规律见附件 1. 基于题目文件附录 2 给出的常数物性参数及水分扩散系数经验公式, 建立预热平衡阶段药材内部温度与水分浓度随时间和径向位置变化的热质传递模型, 仿真时长取 1800 s. 要求在特征时刻 (0-1800s) 与距中心 0-2cm 的特征径向位置处输出温度与水分浓度, 同时以 1 s 时间步长、0.1 cm 径向步长输出全域时空分布由此计算得到近似结果.

问题二: 实际烘干过程通常持续 2-3 天, 预热平衡与恒温干燥阶段的热物性参数存在显著差异. 为贴近实际工况, 采用题目文件附录 3 中随药材温度与含水率动态变化的经验公式, 构建温度—水分双向耦合的热质传递模型, 对包含预热平衡与恒温干燥的完整烘干过程进行数值模拟. 要求输出 3 h 内每隔 0.5 h、距中心 0-2cm 各特征径向位置处的温度与水分浓度, 并以1 s 时间步长、0.1 cm 径向步长输出全域时空分布.

问题三: 以药材内部所有位置干基含水率均低于 $0.15\,\mathrm{kg\,kg^{-1}}$ 作为烘干合格判据, 基于问题二建立的耦合模型, 以药材全域最大含水率为监测指标, 确定烘干过程所需时间. 要求输出烘干过程中每隔 6 h、径向每隔 0.5 cm 处的水分浓度, 并以 60 s 、时间步长0.1 cm 径向步长输出全域水分浓度分布.

问题四: 实际干燥过程中药材因水分流失产生尺寸收缩, 几何边界随干燥进程动态变化. 结合附件 2 给出的药材半径随时间变化的数据, 采用题目文件附录 4 的经验公式, 建立几何收缩条件下的热质传递模型, 重新求解满足烘干合格判据所需的烘干时长. 要求输出每隔 6 h、径向每隔 0.5 cm 及药材表面位置的水分浓度, 并以 60 s 时间步长、0.1 cm 径向步长输出全域水分浓度分布.

二、 初步分析

2.1 问题一的分析

问题一要求建立预热平衡阶段药材温度和水分浓度变化规律的数学模型.

药材视为圆柱形, 长 25 cm, 半径 2 cm. 我们以药材几何中心为原点建立柱坐标系, 同时考虑径向和轴向的传递过程, 建立二维轴对称模型. 传热方面, 取环形微元进行能量守

恒分析, 结合傅里叶定律与比热容公式, 得到二维轴对称导热方程; 水分方面, 取同一微元进行质量守恒分析, 得到含水率依赖的二维轴对称扩散方程.

边界条件方面, 圆柱中心采用对称条件, 径向外表面和轴向端面均与烘房空气发生对流换热和对流传质. 表面水分蒸发会吸收汽化潜热, 因此传热边界中必须加入蒸发吸热项. 初始温度为 28 °C, 初始干基含水率为 $2.55\,\mathrm{kg\,kg^{-1}}$. 求解采用有限体积法离散空间, 隐式时间积分处理刚性. 进一步详细可见 [5, 6, 7].

2.2 问题二、三、四的分析

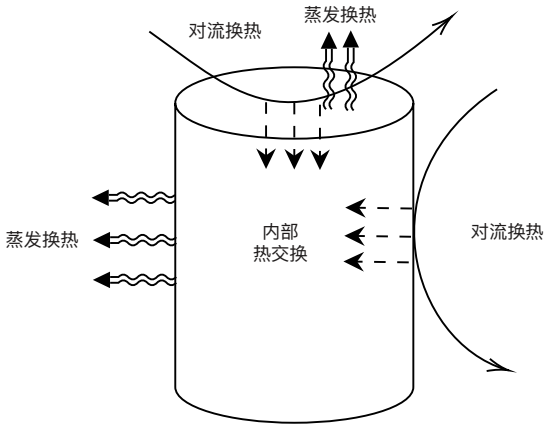
问题二、三、四均是在问题一基础上的进一步推广, 控制方程的基本结构不变, 差别体现在参量形式、输出目标和边界条件上.

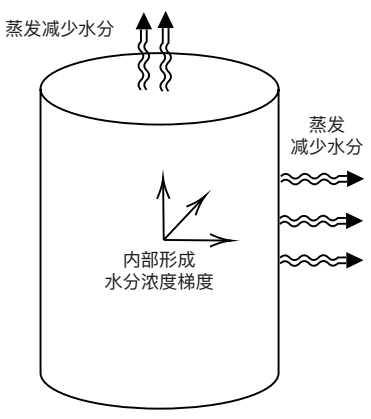
问题二将常参量推广为变参量. 密度、比热容、热传导系数均随含水率变化, 扩散系数同时依赖含水率和温度, 经验公式统一采用题目文件附录 3. 因此问题二的核心是温湿耦合: 含水率影响热物性, 温度通过扩散系数影响水分迁移, 控制方程中的物性系数必须保留在散度算子内部. 烘干过程分为预热平衡和恒温干燥两个阶段, 但两阶段控制方程和边界条件形式一致, 差异只体现在环境参数上.

问题三将输出目标从特定时刻的温度和含水率推广为烘干总时长. 由于水分由内部向表面迁移, 中心是水分最难排出的位置, 本文假设全域最大含水率始终出现在圆柱体中心, 烘干结束判据简化为中心含水率首次低于 $0.15\,\mathrm{kg\,kg^{-1}}$ 的时刻. 问题三虽然只输出水分浓度, 但扩散系数依赖温度, 温度场仍需同时求解.

问题四将固定边界推广为移动边界. 由于半径和高度随时间变化, 引入参考坐标将移动区域映射到固定区域. 坐标变换后, 时间导数会产生移动坐标项, 空间扩散项会出现几何尺度因子. 附件 2 只给出半径数据, 便于思考, 我们假设药材等比例收缩. 物性公式改用题目文件附录 4 后, 仍需保留在散度算子内部. 烘干判据与问题三相同, 输出时需将参考坐标映射回物理坐标, 超出当前半径的位置不再输出.

读者可参考文献 [1, 2, 3, 4] 了解更多相关知识.





三、 模型假设

为了适当简化模型, 我们给出如下总假设.

假设 1 药材在整个干燥过程中没有与其他物体的接触面, 即可以视作“悬浮”在空中.

假设 2 烘房内气压可视作为一常值 $P_a = 101\,325\,\mathrm{Pa}$, 即标准大气压.

假设 3 烘房内各处水汽与温度均匀分布, 即我们忽略不计距离药材的远近与含水量、温度之间的关系.

假设 4 对于药材与空气发生对流热交换时, 我们不计发生热交换的薄壳厚度.

假设 5 对于药材与空气接触面, 我们只考虑水分蒸发带来的热交换以及对流热交换. 同时蒸发热交换我们假设发生在最外层不计厚度的薄壳上, 即水分不在内部蒸发. 同时, 我们认为药材表层水汽饱和.

假设 6 水的气化潜热视作一常数 $L_v = 2.45 \times 10^6\,\mathrm{J\,kg^{-1}}$

假设 7 药材各方向上的比热容、密度、热传导系数和水扩散系数在各方向上相同, 不受地球引力方向与大小的影响, 即不随空间方向变化而变化.

假设 8 恒温干燥阶段, 环境条件通过算法*6*取附件 *1* 末端一小时内稳定值, 即温度$50\,^\circ\mathrm{C}$、干基含水率$0.049\,97\,\mathrm{kg\,kg^{-1}}$

四、 符号说明

符号	含义	单位
$T_s(t), T_a(t)$	t 时刻药材表面温度,t 时刻烘房环境温度	K
$T(t, r, z)$	t 时刻药材在环 (r, z) 上的温度	K
P_a	烘房内气压	Pa
t	烘干时间	s
M	药材水分质量	kg
$R(t), R_0$	药材收缩过程中的瞬时半径, 药材的初始半径	m
$L(t), L_0$	药材收缩过程中的瞬时高度, 药材的初始高度	m
r	药材中截面上的径向距离	m
l, z	距离中心截面的轴向距离, 沿药材轴线的坐标	m
C, C_a	药材水分浓度 (干基含水率), 烘房环境水分浓度	$\mathrm{kg\,kg^{-1}}$
D	水分浓度扩散系数	$\mathrm{m^2\,s^{-1}}$
ρ	药材密度	$\mathrm{kg\,m^{-3}}$
c_p	药材比热容	$\mathrm{J\,kg^{-1}\,K^{-1}}$
k	药材热传导系数	$\mathrm{W\,m^{-1}\,K^{-1}}$
h	对流换热系数	$\mathrm{W\,m^{-2}\,K^{-1}}$
h_m	对流传质系数	$\mathrm{m\,s^{-1}}$
γ	收缩比,$\gamma = R(t)/R_0$	-
Δt	时间步长	s
ρ_s, ρ_a	药材表面水汽质量浓度, 烘房水汽质量浓度,	$\mathrm{kg\,m^{-3}}$
n''_A, n_A	水汽总质量通量, 水汽总质量流量	$\mathrm{kg\,m^{-2}\,s^{-1}}$, $\mathrm{kg\,s^{-1}}$
Q	药材积累的热量	J
q_r, q_z	径向热流量, 轴向热流量	W
n_r, n_z	径向水质量流量, 轴向水质量流量	$\mathrm{kg\,s^{-1}}$
L_v	水的气化潜热	$\mathrm{J\,kg^{-1}}$
T_{ss}, C_{ss}	很稳阶段稳态温度, 稳态湿度	K,$\mathrm{kg\,kg^{-1}}$
$\hat{\cdot}$	坐标变换后的各种量	-

五、 模型的建立与求解

5.1 问题一的模型建立与求解

本文中到药材中心距离均指到药材中心轴的径向距离, 记作 r. 我们以药材中心点为原点, 建立关于径向截面与轴向的二维模型, 如图1所示.

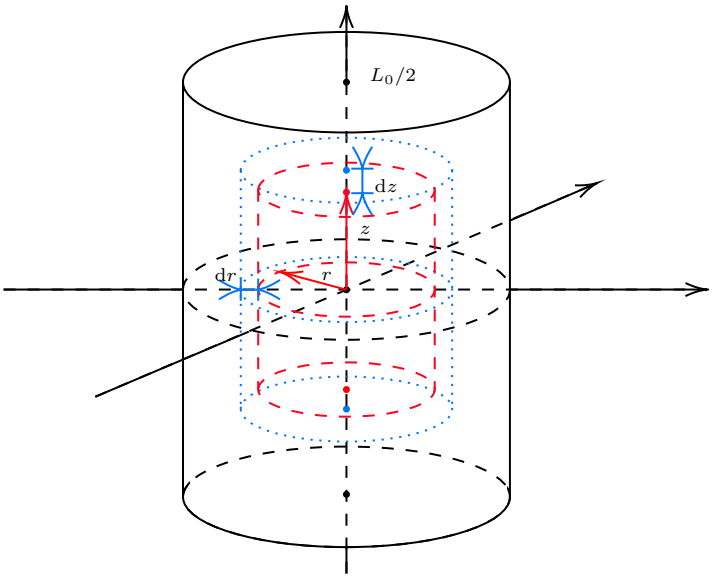


图 **1**　药材内部热传导建模示意图

本问题最后所求的药材温度与水分浓度关于到药材中心的距离均取过中心原点的径向截面上的数据, 取时间步 $\Delta t = 0.5$.

我们首先对内部热传导进行微元法分析, 取微元 $\mathrm{d}r$ 与 $\mathrm{d}z$ 如图2所示, 继而我们得到

$$A_r = 2\pi r\mathrm{d}r, \quad A_z = 2\pi r\mathrm{d}z.$$

其中 A_r 表示圆环微元面积, A_z 表示圆柱侧表面微元面积. 将圆环面积微元 A_r 与轴向厚度微元 d_z 相乘便得到圆柱薄壳微元体积与微元质量

$$\mathrm{d}V = A_r\mathrm{d}z = 2\pi r\mathrm{d}r\mathrm{d}z, \quad \mathrm{d}m = \rho\mathrm{d}V = 2\pi r\rho\mathrm{d}r\mathrm{d}z.$$

由比热容热量公式可知

$$\mathrm{d}Q = c_p\mathrm{d}m\mathrm{d}T = \rho c_p\mathrm{d}T\mathrm{d}V.$$

我们考虑单位时间内微元储能增加速率, 对上式两端除以 $\mathrm{d}t$, 得

$$\frac{\mathrm{d}Q}{\mathrm{d}t} = \rho c_p\frac{\partial T}{\partial t}\mathrm{d}V = 2\pi r\rho c_p\mathrm{d}r\mathrm{d}z\frac{\partial T}{\partial t}. \tag{1}$$

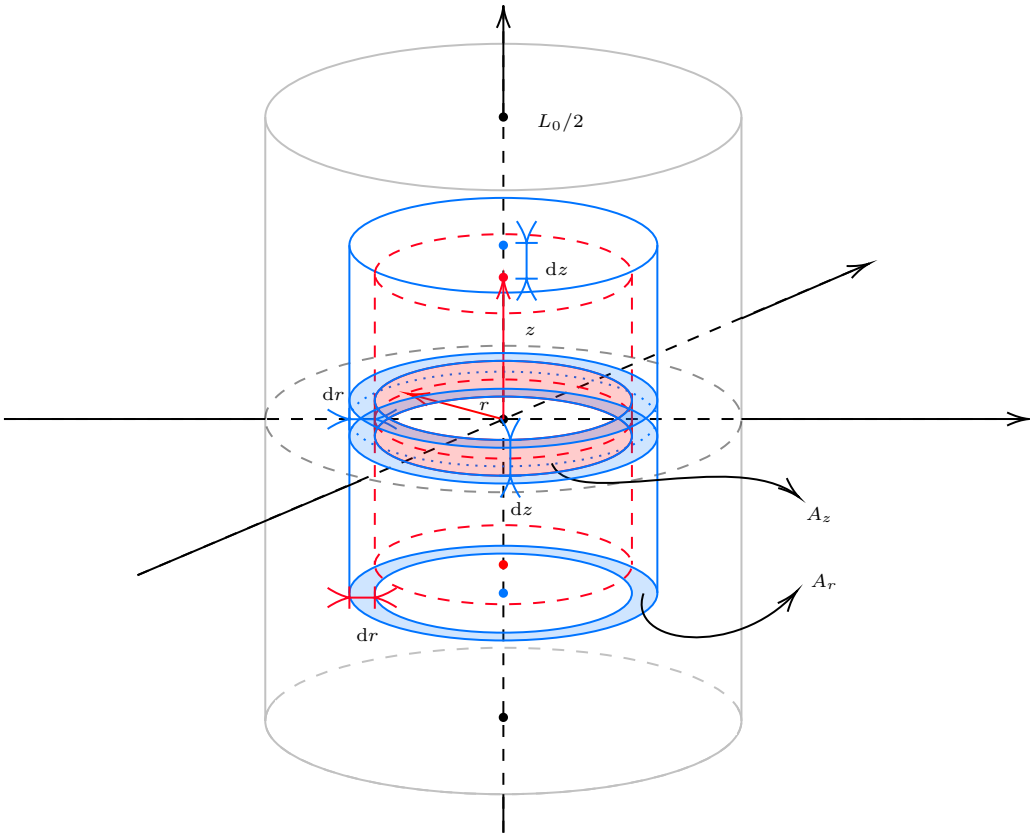


图 **2**　药材内部热传导微元法示意图

同时我们注意到

$$\frac{\mathrm{d}Q}{\mathrm{d}t} = \left(q_r|_r - q_r|_{r+\mathrm{d}r}\right) + \left(q_z|_z - q_z|_{z+\mathrm{d}z}\right). \tag{2}$$

其中

$$q_r|_{r+\mathrm{d}r} - q_r|_r = \frac{\partial q_r}{\partial r}\mathrm{d}r \tag{3}$$

表示沿径向 (向外) 的微元储能增加速率,

$$q_z|_{z+\mathrm{d}z} - q_z|_z = \frac{\partial q_z}{\partial z}\mathrm{d}z \tag{4}$$

表示沿轴向 (向外) 的微元储能增加速率.

结合(1)、(2)、(3)、(4)式, 得

$$-2\pi r\rho c_p \mathrm{d}r\mathrm{d}z\frac{\partial T}{\partial t} = \frac{\partial q_r}{\partial r}\mathrm{d}r + \frac{\partial q_z}{\partial z}\mathrm{d}z. \tag{5}$$

由傅立叶导热定律得到沿径向穿过侧壁 A_z 的热流量为

$$q_r = \left(-k\frac{\partial T}{\partial r}\right)A_z = \left(-k\frac{\partial T}{\partial r}\right)\cdot 2\pi r\mathrm{d}z,$$

沿轴向穿过环形截面 A_r 的热流量为

$$q_z = \left(-k\frac{\partial T}{\partial z}\right) A_r = \left(-k\frac{\partial T}{\partial z}\right) \cdot 2\pi r \mathrm{d}r.$$

将 q_r, q_z 带入(5)式中得

$$\begin{aligned} -2\pi r\rho c_p \frac{\partial T}{\partial t}\mathrm{d}r\mathrm{d}z &= \frac{\partial}{\partial r}\left(-2\pi r k\frac{\partial T}{\partial r}\mathrm{d}z\right)\mathrm{d}r + \frac{\partial}{\partial z}\left(-2\pi r k\frac{\partial T}{\partial z}\mathrm{d}r\right)\mathrm{d}z \\ &= -2\pi\frac{\partial}{\partial r}\left(rk\frac{\partial T}{\partial r}\right)\mathrm{d}r\mathrm{d}z - 2\pi r\frac{\partial}{\partial z}\left(k\frac{\partial T}{\partial z}\right)\mathrm{d}r\mathrm{d}z. \end{aligned}$$

两边同时除去 $-2\pi r\mathrm{d}r\mathrm{d}z$ 得到

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rk\frac{\partial T}{\partial r}\right) + \frac{\partial}{\partial z}\left(k\frac{\partial T}{\partial z}\right). \tag{6}$$

负号表示热量沿着径向的反方向传播. 就问题一而言, ρ, c_p, k 取到常数 $820, 2600, 0.36$, 故(6)在这里可以进一步化简得到

$$\frac{1}{\alpha}\frac{\partial T}{\partial t} = \frac{1}{r}\left(\frac{\partial T}{\partial r} + r\frac{\partial^2 T}{\partial r^2}\right) + \frac{\partial^2 T}{\partial z^2} = \frac{1}{r}\frac{\partial T}{\partial r} + \frac{\partial^2 T}{\partial r^2} + \frac{\partial^2 T}{\partial z^2}, \quad \alpha = \frac{k}{\rho c_p}. \tag{7}$$

同样的, 我们来处理内部水扩散的问题, 方法与内部热扩散是类似的. 我们仍用之前的微元法, 微元内水分质量变化量

$$\mathrm{d}M = \mathrm{d}C\mathrm{d}m = \rho\mathrm{d}C\mathrm{d}V \quad \Longrightarrow \quad \frac{\mathrm{d}M}{\mathrm{d}t} = \rho\frac{\partial C}{\partial t}\mathrm{d}V = 2\pi r\rho\frac{\partial C}{\partial t}\mathrm{d}r\mathrm{d}z. \tag{8}$$

另一方面,

$$\frac{\mathrm{d}M}{\mathrm{d}t} = \left(n_r|_r - n_r|_{r+\mathrm{d}r}\right) + \left(n_z|_z - n_z|_{z+\mathrm{d}z}\right) = -\frac{\partial n_r}{\partial r}\mathrm{d}r - \frac{\partial n_z}{\partial z}\mathrm{d}z. \tag{9}$$

其中 n_r, n_z 分别表示穿过 A_z 的径向水流质量流量与穿过 A_r 轴向水流质量流量.

$$n_r = \left(-\rho D\frac{\partial C}{\partial r}\right) \cdot 2\pi r\mathrm{d}z, \quad n_z = \left(-\rho D\frac{\partial C}{\partial z}\right) \cdot 2\pi r\mathrm{d}r. \tag{10}$$

联立(8)、(9)、(10)式, 得

$$2\pi r\rho\frac{\partial C}{\partial t}\mathrm{d}r\mathrm{d}z = -\frac{\partial}{\partial r}\left(-\rho D\frac{\partial C}{\partial r}\cdot 2\pi r\mathrm{d}z\right)\mathrm{d}r - \frac{\partial}{\partial z}\left(-\rho D\frac{\partial C}{\partial z}\cdot 2\pi r\mathrm{d}r\right)\mathrm{d}z.$$

整理后得

$$\rho\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(\rho r D\frac{\partial C}{\partial r}\right) + \frac{\partial}{\partial z}\left(\rho D\frac{\partial C}{\partial z}\right). \tag{11}$$

就问题一而言, (11)式中的 ρ 是一个常数, 可以省去, 即化为

$$\frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left(rD\frac{\partial C}{\partial r}\right)+\frac{\partial}{\partial z}\left(D\frac{\partial C}{\partial z}\right). \tag{12}$$

现在我们处理药材边界上的热交换, 即给出边界条件如下

$$\begin{cases}\left.\dfrac{\partial T}{\partial r}\right|_{r=0}=0, & -\dfrac{L_0}{2}\leqslant z\leqslant\dfrac{L_0}{2},\\ \left.-k\,\dfrac{\partial T}{\partial r}\right|_{r=R_0}=h\big(T_s(t)-T_a(t)\big)+h_m(\rho_s-\rho_a)L_v, & -\dfrac{L_0}{2}\leqslant z\leqslant\dfrac{L_0}{2},\\ \left.k\,\dfrac{\partial T}{\partial z}\right|_{z=-L_0/2}=h\big(T_s(t)-T_a(t)\big)+h_m(\rho_s-\rho_a)L_v, & 0\leqslant r\leqslant R_0,\\ \left.-k\,\dfrac{\partial T}{\partial z}\right|_{z=L_0/2}=h\big(T_s(t)-T_a(t)\big)+h_m(\rho_s-\rho_a)L_v, & 0\leqslant r\leqslant R_0.\end{cases} \tag{13}$$

同样的给出边界上水交换, 即给出边界条件如下

$$\begin{cases}\left.\dfrac{\partial C}{\partial r}\right|_{r=0}=0, & -\dfrac{L_0}{2}\leqslant z\leqslant\dfrac{L_0}{2},\\ \left.-D\,\dfrac{\partial C}{\partial r}\right|_{r=R_0}=h_m(\rho_s-\rho_a), & -\dfrac{L_0}{2}\leqslant z\leqslant\dfrac{L_0}{2},\\ \left.D\,\dfrac{\partial C}{\partial z}\right|_{z=-L_0/2}=h_m(\rho_s-\rho_a), & 0\leqslant r\leqslant R_0,\\ \left.-D\,\dfrac{\partial C}{\partial z}\right|_{z=L_0/2}=h_m(\rho_s-\rho_a), & 0\leqslant r\leqslant R_0.\end{cases} \tag{14}$$

其中 ρ_s 和 ρ_a 是药材表面水汽质量浓度和烘房水汽质量浓度, 具体的为

$$\rho_s=\frac{M_v}{R_uT_s}\cdot A\exp\left(\frac{B(T_s-273.15)}{C+(T_s-273.15)}\right),\quad \rho_a=C_a\frac{P_a}{R_aT_a}.$$

其中水的摩尔质量 $M_v=0.018\,015\,\mathrm{kg\,mol^{-1}}$, 通用气体常数 $R_u=8.314\,472\,\mathrm{J\,mol^{-1}\,K^{-1}}$, 饱和水汽压系数 $A=611.2\,\mathrm{Pa}$, 温度系数 $B=17.62$, 温度补偿常数 $C=243.12\,^\circ\mathrm{C}$, 干空气气体常数 $R_a=287.05\,\mathrm{J\,kg^{-1}\,K^{-1}}$.

至此, 我们给出了适合问题一的方程组

温度方程(7)及其边界条件(13); 水分方程(12)及其边界条件(14).

为了简化方程组的求解, 需要对饱和水汽密度函数进行数值简化, 我们采用三次多项式拟合的方法, 详见算法1

现在我们可以给出并通过算法2、3、4、5得到问题一方程组的数值解. 所得结果保存在对应文件中, 主要结果在表1、2中展示.

表 **1**　**30** 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.048	28.058	28.072	28.072	28.094
300	28.308	28.337	28.376	28.377	28.439
600	29.033	29.082	29.148	29.148	29.253
900	30.081	30.145	30.232	30.232	30.369
1200	31.348	31.426	31.529	31.530	31.694
1500	32.759	32.841	32.949	32.950	33.123
1800	34.224	34.311	34.426	34.427	34.611

表 **2**　**30** 分钟内药材的水分浓度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5369	2.5370	2.5349	2.5327	2.5081
300	2.5140	2.5139	2.5120	2.5098	2.4849
600	2.4799	2.4798	2.4779	2.4758	2.4511
900	2.4463	2.4462	2.4443	2.4422	2.4179
1200	2.4131	2.4130	2.4112	2.4091	2.3851
1500	2.3804	2.3803	2.3785	2.3765	2.3528
1800	2.3482	2.3481	2.3463	2.3443	2.3210

Algorithm 1: 饱和水汽密度三次多项式拟合

Input: 温度区间 $T_{\min} = 20\,^{\circ}\mathrm{C}, T_{\max} = 50\,^{\circ}\mathrm{C}$
水的摩尔质量 $M_v = 0.018\,015\,\mathrm{kg\,mol^{-1}}$
通用气体常数 $R_u = 8.314\,472\,\mathrm{J\,mol^{-1}\,K^{-1}}$
Magnus 公式系数 $A = 6.112\,\mathrm{hPa},\ B = 17.62,\ C = 243.12\,^{\circ}\mathrm{C}$
Output: 中心化三次多项式系数 a_3, a_2, a_1, a_0
拟合最大相对误差 $\max(err)$
/* 生成热力学温度采样点 */
$T_s \leftarrow \mathrm{Linspace}(293.15,\ 323.15,\ 1000)$
/* Magnus 公式计算原始饱和水汽密度 */
$$\rho_s \leftarrow \frac{M_v \cdot A \exp\left(\dfrac{B(T_s - 273.15)}{C + T_s - 273.15}\right) \cdot 100}{R_u T_s}$$
/* 温度中心化：中心温度 $308.15\,\mathrm{K}$ $(35\,^{\circ}\mathrm{C})$ */
$x \leftarrow T_s - 308.15$
/* 三次多项式拟合 */
$(a_3, a_2, a_1, a_0) \leftarrow \mathrm{Polyfit}(x,\ \rho_s,\ 3)$
/* 计算拟合值与相对误差 */
$$\rho_{\mathrm{fit}} \leftarrow a_3x^3 + a_2x^2 + a_1x + a_0 \quad err \leftarrow \left|\frac{\rho_{\mathrm{fit}} - \rho_s}{\rho_s}\right| \times 100\%$$
return $a_3, a_2, a_1, a_0,\ \max(err)$

Algorithm 2: 药材温度场与水分场有限体积求解 (数据预处理)

Input: 环境数据时间序列 $(t_k^{env}, T_k^{env}, C_k^{env})$, 来自附件 1
几何参数 R, L, 网格参数 N_r, N_z
温度参数 $\rho, c_p, k, h, T_0, \Delta t_T$
水分参数 D_0, α, h_m, C_0
终止时间 t_{end}
输出时间网格 t_{all}, 输出半径网格 r_{all}
Output: 温度场离散结果 $\{T^{(m)}(r_i, z_j)\}$ 与水分场离散结果 $\{C^{(m)}(r_i, z_j)\}$
固定 $z = 0$ 截面上的表 1 与表 2
`result1.xlsx` 中“温度”与“水分浓度”工作表
/* 公共初始化 */
读取附件 1, 得到 $t^{env}, T^{env}, C^{env}$
$T^{env} \leftarrow T^{env} + 273.15$; // 温度转换为开尔文
构造插值函数: $T_{\mathrm{env}}(t) \leftarrow \mathrm{Interp1D}(t^{env}, T^{env})$
$C_{\mathrm{env}}(t) \leftarrow \mathrm{Interp1D}(t^{env}, C^{env})$
$R \leftarrow 0.02,\ L \leftarrow 0.25,\ N_r \leftarrow 40,\ N_z \leftarrow 80$
$\Delta r \leftarrow R/N_r,\ \Delta z \leftarrow L/N_z,\ N \leftarrow N_rN_z$
$r_i \leftarrow (i + 0.5)\Delta r,\ i = 0, \ldots, N_r - 1$
$z_j \leftarrow -L/2 + (j + 0.5)\Delta z,\ j = 0, \ldots, N_z - 1$
定义全局索引映射 $n(i, j) \leftarrow i + N_rj$

Algorithm 3: 药材温度场与水分场有限体积求解 (温度场求解)

$\rho \leftarrow 820$, $c_p \leftarrow 2600$, $k \leftarrow 0.36$, $h \leftarrow 25$
$T_0 \leftarrow 301.15$, $\Delta t_T \leftarrow 1.0$
$T_{i,j}^{(0)} \leftarrow T_0, \forall i, j$
$t \leftarrow 0$, $\mathcal{T} \leftarrow [0]$, $\mathcal{T}_{\text{field}} \leftarrow [T^{(0)}]$
for $m \leftarrow 1, 2, \ldots$ 且$t \le t_{\text{end}}$ **do**
  $t \leftarrow t + \Delta t_T$
  $T_\infty \leftarrow T_{\text{env}}(t)$
  /* 组装温度场稀疏线性系统 */
  $A_T \leftarrow \text{ZerosSparse}(N, N)$, $b_T \leftarrow \text{Zeros}(N)$
  for $j \leftarrow 0$ **to** $N_z - 1$ **do**
    for $i \leftarrow 0$ **to** $N_r - 1$ **do**
      $n \leftarrow n(i, j)$
      $V \leftarrow r_i \Delta r \Delta z$
      $r^+ \leftarrow r_i + 0.5\Delta r$, $r^- \leftarrow r_i - 0.5\Delta r$
      if $i = 0$ **then**
        $r^- \leftarrow 0$
      $A_{r^+} \leftarrow r^+ \Delta z$, $A_{r^-} \leftarrow r^- \Delta z$, $A_z \leftarrow r_i \Delta r$
      $a_P \leftarrow \rho c_p V / \Delta t_T$, $b_T(n) \leftarrow a_P T_{i,j}^{(m-1)}$
      /* 径向离散 */
      if $i = N_r - 1$ **then**
        $a_P \leftarrow a_P + kA_{r^+}/\Delta r + hA_{r^+}$
        $b_T(n) \leftarrow b_T(n) + hA_{r^+} T_\infty$
        在 A_T 中加入与 $T_{i-1,j}$ 的耦合项
      else
        if $i < N_r - 1$ **then**
          $a_E \leftarrow kA_{r^+}/\Delta r$
          $a_P \leftarrow a_P + a_E$
          在 A_T 中加入与 $T_{i+1,j}$ 的耦合项
        if $i > 0$ **then**
          $a_W \leftarrow kA_{r^-}/\Delta r$
          $a_P \leftarrow a_P + a_W$
          在 A_T 中加入与 $T_{i-1,j}$ 的耦合项
      /* 轴向离散 */
      if $j = 0$ 或$j = N_z - 1$ **then**
        $a_P \leftarrow a_P + kA_z/\Delta z + hA_z$
        $b_T(n) \leftarrow b_T(n) + hA_z T_\infty$
        在 A_T 中加入与相邻轴向节点的耦合项
      else
        $a_N \leftarrow kA_z/\Delta z$
        $a_S \leftarrow kA_z/\Delta z$
        $a_P \leftarrow a_P + a_N + a_S$
        在 A_T 中加入与 $T_{i,j+1}$ 和 $T_{i,j-1}$ 的耦合项
      $A_T(n, n) \leftarrow a_P$
  /* 求解温度线性系统并更新温度场 */
  $T^{\text{vec}} \leftarrow \text{SparseSolve}(A_T, b_T)$
  $T^{(m)} \leftarrow \text{Reshape}(T^{\text{vec}}, N_r, N_z)$
  将 t 加入 $\mathcal{T}$, 将 $T^{(m)}$ 加入 $\mathcal{T}_{\text{field}}$

Algorithm 4: 药材温度场与水分场有限体积求解 (水分场求解)

$D_0 \leftarrow 7 \times 10^{-9}, \alpha \leftarrow 0.89$
$h_m \leftarrow 8 \times 10^{-7}, C_0 \leftarrow 2.55$
$C_{i,j}^{(0)} \leftarrow C_0, \forall i, j$
$t \leftarrow 0, \mathcal{T}_C \leftarrow [0], \mathcal{C}_{\text{field}} \leftarrow [C^{(0)}]$
for $m \leftarrow 1, 2, \ldots$ 且$t \le t_{\text{end}}$ **do**
  if $t < 1$ **then**
    $\Delta t_C \leftarrow \min(0.1, 1 - t)$; // 第一阶段: 0 1 s, 步长 0.1 s
  else
    $\Delta t_C \leftarrow \min(1.0, t_{\text{end}} - t)$; // 第二阶段: 1 1800 s, 步长 1 s
  if $\Delta t_C \le 0$ **then**
    break
  $t \leftarrow t + \Delta t_C$
  $C_\infty \leftarrow C_{\text{env}}(t)$
  /* 组装水分场稀疏线性系统 */
  $A_C \leftarrow \text{ZerosSparse}(N, N), b_C \leftarrow \text{Zeros}(N)$
  for $j \leftarrow 0$ **to** $N_z - 1$ **do**
    for $i \leftarrow 0$ **to** $N_r - 1$ **do**
      $n \leftarrow n(i, j)$
      $V \leftarrow r_i \Delta r \Delta z$
      $D \leftarrow D_0 \exp(-\alpha / \max(C_{i,j}^{(m-1)}, 10^{-6}))$
      $r^+ \leftarrow r_i + 0.5\Delta r, r^- \leftarrow r_i - 0.5\Delta r$
      if $i = 0$ **then**
        $r^- \leftarrow 0$
      $A_{r+} \leftarrow r^+ \Delta z, A_{r-} \leftarrow r^- \Delta z, A_z \leftarrow r_i \Delta r$
      $a_P \leftarrow V / \Delta t_C, b_C(n) \leftarrow a_P C_{i,j}^{(m-1)}$
      /* 径向离散 */
      if $i = N_r - 1$ **then**
        计算表面质量交换系数
        $a_P \leftarrow a_P + DA_{r+}/\Delta r + h_m A_{r+}$
        $b_C(n) \leftarrow b_C(n) + h_m A_{r+} C_\infty$
        在 A_C 中加入与 $C_{i-1,j}$ 的耦合项
      else
        if $i < N_r - 1$ **then**
          $a_E \leftarrow DA_{r+}/\Delta r$
          $a_P \leftarrow a_P + a_E$
          在 A_C 中加入与 $C_{i+1,j}$ 的耦合项
        if $i > 0$ **then**
          $a_W \leftarrow DA_{r-}/\Delta r$
          $a_P \leftarrow a_P + a_W$
          在 A_C 中加入与 $C_{i-1,j}$ 的耦合项
      /* 轴向离散 */
      if $j = 0$ 或$j = N_z - 1$ **then**
        $a_P \leftarrow a_P + DA_z/\Delta z + h_m A_z$
        $b_C(n) \leftarrow b_C(n) + h_m A_z C_\infty$
        在 A_C 中加入与相邻轴向节点的耦合项
      else
        $a_N \leftarrow DA_z/\Delta z$
        $a_S \leftarrow DA_z/\Delta z$
        $a_P \leftarrow a_P + a_N + a_S$
        在 A_C 中加入与 $C_{i,j+1}$ 和 $C_{i,j-1}$ 的耦合项
      $A_C(n, n) \leftarrow a_P$
  /* 求解水分线性系统并更新水分场 */
  $C^{\text{vec}} \leftarrow \text{SparseSolve}(A_C, b_C)$
  $C^{(m)} \leftarrow \text{Reshape}(C^{\text{vec}}, N_r, N_z)$
  $C^{(m)} \leftarrow \max(C^{(m)}, 10^{-6})$
  将 t 加入 $\mathcal{T}_C$, 将 $C^{(m)}$ 加入 $\mathcal{C}_{\text{field}}$

Algorithm 5: 药材温度场与水分场有限体积求解 (结果处理)

/* 固定 $z = 0$ 截面 */
$j^\star \leftarrow \arg\min_j |z_j - 0|$
$T_m^{\text{slice}}(i) \leftarrow T_{i,j^\star}^{(m)}, \forall m, i$
$C_m^{\text{slice}}(i) \leftarrow C_{i,j^\star}^{(m)}, \forall m, i$
/* 输出表 1 与表 2 */
$t_{\text{target}} \leftarrow [0, 100, 300, 600, 900, 1200, 1500, 1800]$
$r_{\text{target}} \leftarrow [0, 0.5, 1, 1.5, 2]/100$
for 每个目标半径$r_q \in r_{\text{target}}$ **do**
  对每个时刻 t_m, 沿 r 插值得到 $T_r(t_m)$ 与 $C_r(t_m)$
  再沿时间插值到 t_{target}, 得到 T_{show} 与 C_{show}
$T_{\text{show}}(0, :) \leftarrow T_0 - 273.15$
$C_{\text{show}}(0, :) \leftarrow C_0$
输出“表 1 30 分钟内药材的温度”
输出“表 2 30 分钟内药材的水分浓度”
/* 六、写入 Excel */
$t_{\text{all}} \leftarrow 0 : 1 : 1800$
$r_{\text{all}} \leftarrow (0 : 0.1 : 2)/100$
对每个 $r_q \in r_{\text{all}}$, 先沿 r 插值, 再沿时间插值到 t_{all}, 分别得到 T_{result} 与 C_{result}
$T_{\text{result}}(0, :) \leftarrow T_0$
$C_{\text{result}}(0, :) \leftarrow C_0$
打开 `result1.xlsx`
在“温度”工作表中写入时间列与 $T_{\text{result}} - 273.15$
在“水分浓度”工作表中写入时间列与 C_{result}
保存 `result1.xlsx`
return $\mathcal{T}, \mathcal{T}_{\text{field}}, \mathcal{T}_C, \mathcal{C}_{\text{field}}, T_{\text{show}}, C_{\text{show}}, T_{\text{result}}, C_{\text{result}}$

5.2 问题二、三的模型建立与求解

对于恒温过程, 我们给出并通过算法6求得恒温恒湿阶段烘房环境温度与水分浓度稳定值 $T_{ss} \approx 50.00\,^{\circ}\mathrm{C} = 323\,\mathrm{K}$ 与 $C_{ss} \approx 0.049\,97\,\mathrm{kg\,kg^{-1}}$, 如图3、4 所示.

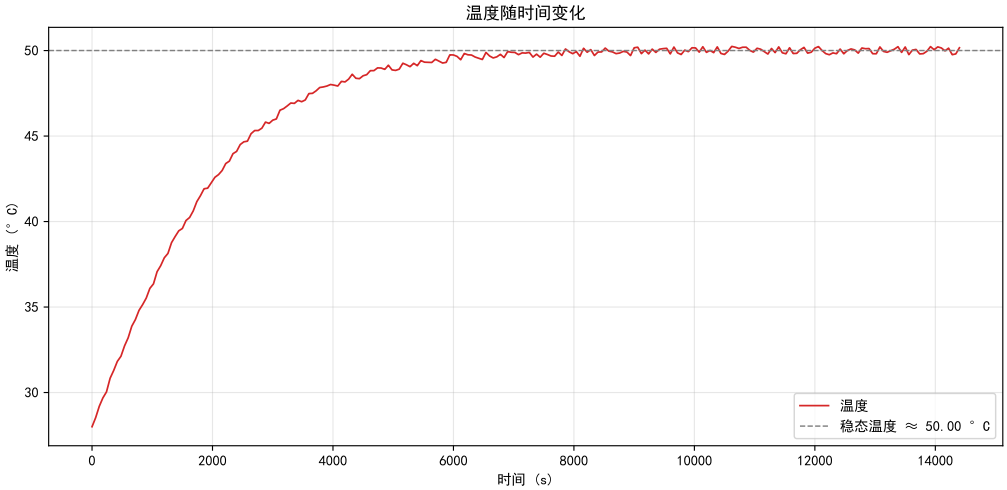


图 **3**　稳态温度示意图

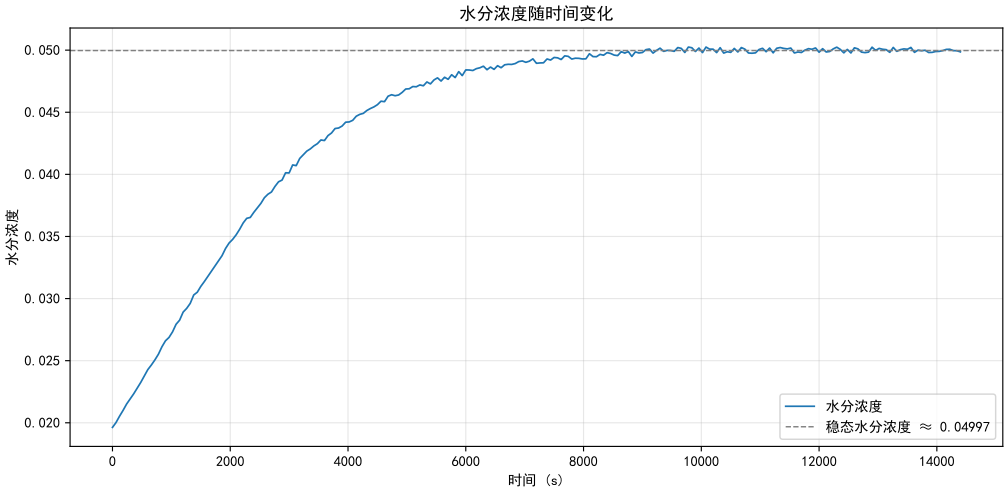


图 **4**　稳态湿度示意图

注意, 对于问题二、三, 此时 ρ, c_p, k 都是 $C = C(r, z, t)$ 的函数, D 是 C, T 的函数, 即

$$\begin{cases} \rho = \rho(C) = 650 + 128C, & c_p = c_p(C) = 1450 + 2736 \cdot \dfrac{C}{C+1}, \\ k = k(C) = 0.21 + 0.38\dfrac{C}{C+1}, & D = D(C,T) = 2.4 \times 10^{-3} \exp\left(-\dfrac{0.45}{C}\right) \exp\left(-\dfrac{3850}{T}\right). \end{cases} \tag{15}$$

我们认为自 3 小时, 即 10800 秒后开始属于稳态恒温恒湿模式, 即

$$T_a(t \geqslant 10800) = 323, \quad \rho_a(t \geqslant 10800) = 0.04997 \times \frac{101325}{287.05 \times 323} = 0.0546. \tag{16}$$

至此, 我们给出了适用于第二问与第三问的主方程组:

参量组(15); 温度方程(6)及其边界条件(13)、水分方程(11)及其边界条件(14).

现在我们可以给出并通过算法7、8、9得到问题二、三方程组的数值解. 所得结果保存在对应文件中, 问题二主要结果在表3、4中展示, 问题三主要结果在表5中展示.

表 **3**　**3** 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.380	32.581	33.190	34.222	35.518
1.0	40.824	40.998	41.507	42.318	43.273
1.5	46.237	46.321	46.566	46.951	47.383
2.0	48.683	48.718	48.817	48.971	49.146
2.5	49.577	49.587	49.614	49.655	49.722
3.0	49.895	49.900	49.917	49.949	49.992

表 **4**　**3** 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5484	2.5217	2.3061	1.6890
1.0	2.5220	2.4885	2.3420	1.9869	1.4782
1.5	2.3697	2.3053	2.1019	1.7512	1.3341
2.0	2.1378	2.0711	1.8742	1.5603	1.1997
2.5	1.9077	1.8475	1.6714	1.3925	1.0693
3.0	1.7033	1.6497	1.4924	1.2415	0.9467

Algorithm 6: 稳态温度与水分浓度计算

Input: 数据文件附件 1.xlsx, 包含时间序列数据 t、温度 T、水汽浓度 C
时间阈值参数 $t_{\max}$
Output: 稳态温度 T_{ss}, 稳态水汽浓度 C_{ss}
T–t、C–t 两组折线图
/* 读取 Excel 时序数据 */
$(t, T, C) \leftarrow$ read_excel("附件 1.xlsx")
/* 选取时间末尾区间的数据点 */
$mask \leftarrow t \geqslant t_{\max} - 1000$; // 截取最后 1000 个数据点
/* 求时间窗口内均值作为稳态值 */
$T_{\mathrm{ss}} \leftarrow \mathrm{mean}(T[mask])$ $C_{\mathrm{ss}} \leftarrow \mathrm{mean}(C[mask])$
/* 绘制时序曲线，叠加稳态水平参考虚线 */
// 绘制 T–t、C–t 折线图；分别叠加 T_{ss}、C_{ss} 水平虚线
return T_{ss}, C_{ss}

Algorithm 7: 药材烘干过程数值模拟与烘干时间确定 (公共初始化: 读取环境数据与插值函数)

Input: 附件 1 环境数据时间序列 $(t_k^{\mathrm{env}}, T_k^{\mathrm{env}}, C_k^{\mathrm{env}})$
几何参数 R, L, 网格参数 N_r, N_z
物性函数 $\rho(C), c_p(C), k(C), D(C,T), \rho_{\mathrm{sat}}(T), \rho_a(T,C)$
换热传质参数 h, h_m, L_v, 初始条件 T_0, C_0
大气压 P_a, 气体常数 R_a, 终止阈值 C_{limit}, 最大模拟时间 $t_{\max}$
Output: result2.xlsx: 前 3 小时每秒温度、水分浓度剖面
result3.xlsx: 烘干至合格每 60 s 水分浓度剖面
烘干结束时刻 t_{finish}
读取附件 1, 得到 $t^{\mathrm{env}}, T^{\mathrm{env}}, C^{\mathrm{env}}$
$T^{\mathrm{env}} \leftarrow T^{\mathrm{env}} + 273.15$; // 转为开尔文温度
构造线性插值: $T_\infty(t) \leftarrow \mathrm{Interp1D}(t^{\mathrm{env}}, T^{\mathrm{env}})$
$C_\infty(t) \leftarrow \mathrm{Interp1D}(t^{\mathrm{env}}, C^{\mathrm{env}})$
$R \leftarrow 0.02$, $L \leftarrow 0.25$, $\Delta r \leftarrow 0.001$, $N_z \leftarrow 10$
$N_r \leftarrow R/\Delta r$, $\Delta z \leftarrow L/N_z$, $N \leftarrow N_r N_z$
$r_i \leftarrow (i + 0.5)\Delta r,\ i = 0, \ldots, N_r - 1$
$z_j \leftarrow -L/2 + (j + 0.5)\Delta z,\ j = 0, \ldots, N_z - 1$
定义索引映射 $\mathrm{idx}(i,j) \leftarrow i + N_r j$
$T_{i,j}^{(0)} \leftarrow T_0,\ C_{i,j}^{(0)} \leftarrow C_0,\ \forall i, j$
$t \leftarrow 0$, $\Delta t \leftarrow 1$, $k \leftarrow 1$
$T_{\mathrm{records}} \leftarrow [\mathrm{Profile}(T^{(0)})]$, $C_{\mathrm{records}} \leftarrow [\mathrm{Profile}(C^{(0)})]$

Algorithm 8: 药材烘干过程数值模拟

for $m \leftarrow 1, 2, \ldots$ 且$t \le 3 \times 3600$ **do**
  $t \leftarrow t + \Delta t$; $T_\infty \leftarrow T_\infty(t)$; $C_\infty \leftarrow C_\infty(t)$; $\rho_a \leftarrow \rho_a(T_\infty, C_\infty)$
  /* 组装温度稀疏线性系统 */
  $A_T \leftarrow \text{ZerosSparse}(N, N)$; $b_T \leftarrow \text{Zeros}(N)$ **for** $j \leftarrow 0$ **to** $N_z - 1$ **do**
    for $i \leftarrow 0$ **to** $N_r - 1$ **do**
      $n \leftarrow \text{idx}(i, j)$; $V \leftarrow r_i \Delta r \Delta z$ $r^+ \leftarrow r_i + 0.5\Delta r$; $r^- \leftarrow r_i - 0.5\Delta r$ **if** $i = 0$ **then**
        $r^- \leftarrow 0$
      $A_{r+} \leftarrow r^+ \Delta z$; $A_{r-} \leftarrow r^- \Delta z$; $A_z \leftarrow r_i \Delta r$ $\rho \leftarrow \rho(C_{i,j}^{(m-1)})$; $c_p \leftarrow c_p(C_{i,j}^{(m-1)})$; $k \leftarrow k(C_{i,j}^{(m-1)})$
       $a_P \leftarrow \rho c_p V / \Delta t$; $b_T(n) \leftarrow a_P\, T_{i,j}^{(m-1)}$
      /* 径向离散，含表面蒸发潜热源项 */
      if $i = N_r - 1$ **then**
        $a_P \leftarrow a_P + kA_{r+}/\Delta r + hA_{r+}$ evap $\leftarrow h_m\big(\rho_{\text{sat}}(T_{i,j}^{(m-1)}) - \rho_a\big) L_v$
         $b_T(n) \leftarrow b_T(n) + hA_{r+} T_\infty -$ evap A_{r+} 在 A_T 加入与 $T_{i-1,j}$ 的耦合项
      else
        if $i < N_r - 1$ **then**
          $a_E \leftarrow kA_{r+}/\Delta r$; $a_P \leftarrow a_P + a_E$; 在 A_T 加入与 $T_{i+1,j}$ 的耦合项
        if $i > 0$ **then**
          $a_W \leftarrow kA_{r-}/\Delta r$; $a_P \leftarrow a_P + a_W$; 在 A_T 加入与 $T_{i-1,j}$ 的耦合项
      /* 轴向离散，端面对流 + 蒸发潜热 */
      if $j = 0$ 或$j = N_z - 1$ **then**
        $a_P \leftarrow a_P + kA_z/\Delta z + hA_z$ evap $\leftarrow h_m\big(\rho_{\text{sat}}(T_{i,j}^{(m-1)}) - \rho_a\big) L_v$
         $b_T(n) \leftarrow b_T(n) + hA_z T_\infty -$ evap A_z 在 A_T 加入与相邻轴向节点的耦合项
      else
        $a_N \leftarrow kA_z/\Delta z$; $a_S \leftarrow kA_z/\Delta z$ $a_P \leftarrow a_P + a_N + a_S$ 在 A_T 加入与 $T_{i,j+1}, T_{i,j-1}$ 的耦合项
      $A_T(n, n) \leftarrow a_P$
  /* 求解温度场 */
  $T^{\text{vec}} \leftarrow \text{SparseSolve}(A_T, b_T)$; $T^{(m)} \leftarrow \text{Reshape}(T^{\text{vec}}, N_r, N_z)$
  /* 组装水分浓度稀疏线性系统 */
  $A_C \leftarrow \text{ZerosSparse}(N, N)$; $b_C \leftarrow \text{Zeros}(N)$ **for** $j \leftarrow 0$ **to** $N_z - 1$ **do**
    for $i \leftarrow 0$ **to** $N_r - 1$ **do**
      $n \leftarrow \text{idx}(i, j)$; $V \leftarrow r_i \Delta r \Delta z$ $D \leftarrow D(C_{i,j}^{(m-1)}, T_{i,j}^{(m)})$ $r^+ \leftarrow r_i + 0.5\Delta r$; $r^- \leftarrow r_i - 0.5\Delta r$ **if** $i = 0$ **then**
        $r^- \leftarrow 0$
      $A_{r+} \leftarrow r^+ \Delta z$; $A_{r-} \leftarrow r^- \Delta z$; $A_z \leftarrow r_i \Delta r$ $a_P \leftarrow V/\Delta t$; $b_C(n) \leftarrow a_P\, C_{i,j}^{(m-1)}$
      /* 径向离散，表面对流传质 */
      if $i = N_r - 1$ **then**
        $a_P \leftarrow a_P + DA_{r+}/\Delta r + h_m A_{r+}$ $b_C(n) \leftarrow b_C(n) + h_m A_{r+}\big(\rho_{\text{sat}}(T_{i,j}^{(m)}) - \rho_a\big)$ 在 A_C 加入与 $C_{i-1,j}$ 的耦合项
      else
        if $i < N_r - 1$ **then**
          $a_E \leftarrow DA_{r+}/\Delta r$; $a_P \leftarrow a_P + a_E$; 在 A_C 加入与 $C_{i+1,j}$ 的耦合项
        if $i > 0$ **then**
          $a_W \leftarrow DA_{r-}/\Delta r$; $a_P \leftarrow a_P + a_W$; 在 A_C 加入与 $C_{i-1,j}$ 的耦合项
      /* 轴向离散，端面对流传质 */
      if $j = 0$ 或$j = N_z - 1$ **then**
        $a_P \leftarrow a_P + DA_z/\Delta z + h_m A_z$ $b_C(n) \leftarrow b_C(n) + h_m A_z\big(\rho_{\text{sat}}(T_{i,j}^{(m)}) - \rho_a\big)$ 在 A_C 加入与相邻轴向节点的耦合项
      else
        $a_N \leftarrow DA_z/\Delta z$; $a_S \leftarrow DA_z/\Delta z$ $a_P \leftarrow a_P + a_N + a_S$ 在 A_C 加入与 $C_{i,j+1}, C_{i,j-1}$ 的耦合项
      $A_C(n, n) \leftarrow a_P$
  /* 求解水分场 */
  $C^{\text{vec}} \leftarrow \text{SparseSolve}(A_C, b_C)$; $C^{(m)} \leftarrow \text{Reshape}(C^{\text{vec}}, N_r, N_z)$; $C^{(m)} \leftarrow \max(C^{(m)}, 10^{-6})$
  /* 记录每秒剖面；定时输出表格点位 */
  if $k \le 10800$ **and** $t \ge k$ **then**
    $T_{\text{records}}.\text{Append}\big(\text{Profile}(T^{(m)}) - 273.15\big)$; $C_{\text{records}}.\text{Append}\big(\text{Profile}(C^{(m)})\big)$; $k \leftarrow k + 1$
  if $t \in \{1800, 3600, 5400, 7200, 9000, 10800\}$ **then**
    输出表 3、表 4 径向点位: 0, 0.5, 1.0, 1.5, 2.0 cm
/* 输出第一阶段结果至 `result2.xlsx` */
打开 `result2.xlsx`, 写入 T_{records}(保留 3 位小数)、C_{records}(保留 4 位小数); 保存 `result2.xlsx`
return t_{finish}

Algorithm 9: 药材烘干过程烘干时间确定

从 `result2.xlsx` 读取 $t = 10800\,\mathrm{s}$ 剖面, 插值到网格, 轴向扩展得到 T, C
$C \leftarrow \max(C, 10^{-6})$
$T_\infty \leftarrow 50 + 273.15$, $C_\infty \leftarrow 0.04997$
$\rho_a \leftarrow \rho_a(T_\infty, C_\infty)$, $C_{\text{limit}} \leftarrow 0.15$
$t \leftarrow 10800$, $\Delta t \leftarrow 1$, $t_{\text{record}} \leftarrow 10860$, $t_{\text{table}} \leftarrow 6 \times 3600$
$C_{\text{records2}} \leftarrow []$, $t_{\text{finish}} \leftarrow$ None
for $m \leftarrow 1, 2, \ldots$ 且$t \le t_{\max}$ **do**
  $t \leftarrow t + \Delta t$
  /* 调用组装函数求解温度、水分场 */
  $(A_T, b_T) \leftarrow \text{AssembleT}(T, T_\infty, C, \rho_a, \Delta t)$
  $T^{\text{vec}} \leftarrow \text{SparseSolve}(A_T, b_T)$; $T \leftarrow \text{Reshape}(T^{\text{vec}}, N_r, N_z)$
  $(A_C, b_C) \leftarrow \text{AssembleC}(C, C_\infty, T, T_\infty, \Delta t)$
  $C^{\text{vec}} \leftarrow \text{SparseSolve}(A_C, b_C)$; $C \leftarrow \text{Reshape}(C^{\text{vec}}, N_r, N_z)$
  $C \leftarrow \max(C, 10^{-6})$
  /* 烘干终止判据 */
  if $\max(C) < C_{\text{limit}}$ **then**
    $t_{\text{finish}} \leftarrow t$; **break**
  /* 每 `60s` 记录剖面; 每 `6h` 输出表 5 点位 */
  if $t \ge t_{\text{record}}$ **then**
    $C_{\text{records2}}.\text{Append}\big(\text{Profile}(C, 0 : 0.1 : 2.0)\big)$
    $t_{\text{record}} \leftarrow t_{\text{record}} + 60$
  if $t \ge t_{\text{table}}$ **then**
    输出表 5 径向点位: $0, 0.5, 1.0, 1.5, 2.0$ cm
    $t_{\text{table}} \leftarrow t_{\text{table}} + 6 \times 3600$
/* 输出第二阶段结果至 `result3.xlsx` */
打开 `result3.xlsx`, 写入 C_{records2}(保留 4 位小数)
保存 `result3.xlsx`
return t_{finish}

表 **5**　药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	0.9015	0.8689	0.7716	0.6092	0.4012
12	0.3208	0.3049	0.2567	0.1730	0.0281
18	0.2101	0.2015	0.1750	0,1264	0.0281
24	0.1726	0.1663	0.1467	0.1097	0.0281
30	0.1531	0.1480	0.1318	0.1006	0.0281
31.313	0.1500	0.1450	0.1293	0.0991	0.0281

5.3 问题四的模型建立与求解

对于边界收缩问题, 我们提出如下假设

假设 9 *问题 4 中收缩视作均匀收缩, 即所有点按同一比例系数 $\gamma = R(t)/R_0$ 收缩.*

控制方程仍沿用(6)与(11)式, 参量组改为如下(17)

$$\begin{cases} \rho(C) = 760 + 90C, & c_p(C) = 1850 + 2150\dfrac{C}{C+1}, \\ k(C) = 0.12 + 0.20\dfrac{C}{C+1}, & D(C,T) = 4.2\times 10^{-4}\exp\left(-\dfrac{0.30}{C}\right)\exp\left(-\dfrac{3850}{T}\right). \end{cases} \tag{17}$$

收缩过程中药材区域可以描述为

$$\Omega(t) = \left\{(r,z) : 0 \leqslant r \leqslant R(t),\ -\frac{L(t)}{2} \leqslant z \leqslant \frac{L(t)}{2}\right\}.$$

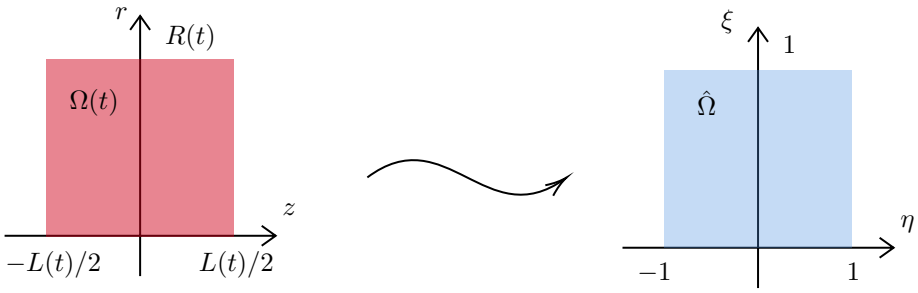
同时有 $R(t) = \gamma(t)R_0, L(t) = \gamma(t)L_0$.

定义无量纲坐标

$$\xi = \frac{r}{R(t)}, \quad \eta = \frac{2z}{L(t)}, \qquad 0 \leq \xi \leq 1, \quad -1 \leq \eta \leq 1.$$

反变换为 $r = R(t)\xi, z = L(t)\eta/2$. 于是任意时刻移动物理域 $\Omega(t)$ 被映射到固定矩形域

$$\hat{\Omega} = \{(\xi,\eta) : 0 \leqslant \xi \leqslant 1,\ -1 \leqslant \eta \leqslant 1\}.$$



定义变换后的温度、水分场

$$\hat{T}(\xi,\eta,t) = T\left(R(t)\xi, \frac{L(t)}{2}\eta, t\right), \quad \hat{C}(\xi,\eta,t) = C\left(R(t)\xi, \frac{L(t)}{2}\eta, t\right).$$

映射 $(\xi,\eta) \mapsto (r,z)$ 的二维雅可比矩阵及其行列式为

$$\mathbf{J} = \begin{pmatrix} \partial r/\partial\xi & \partial r/\partial\eta \\ \partial z/\partial\xi & \partial z/\partial\eta \end{pmatrix} = \begin{pmatrix} R(t) & 0 \\ 0 & L(t)/2 \end{pmatrix}, \quad |\mathbf{J}| = \frac{R(t)L(t)}{2}.$$

对于轴对称三维圆柱, 物理体积元

$$\mathrm{d}V = 2\pi r\,\mathrm{d}r\,\mathrm{d}z = 2\pi(R\xi)(R\,\mathrm{d}\xi)\left(\frac{L}{2}\mathrm{d}\eta\right) = \pi R^2(t)L(t)\xi\,\mathrm{d}\xi\,\mathrm{d}\eta.$$

在后文的推导中, 为了书写的简洁与阅读的可行性, 下标表示偏导, $\dot{\bullet}$ 表示一阶导.
由 $\xi = r/R(t)$, 固定 r 时有

$$\left.\frac{\partial\xi}{\partial t}\right|_r = \frac{\partial}{\partial t}\left(\frac{r}{R(t)}\right)\Bigg|_r = -\frac{r\dot{R}(t)}{R^2(t)} = -\frac{\dot{R}(t)}{R(t)}\xi.$$

同理由 $\eta = 2z/L(t)$, 固定 z 时有

$$\left.\frac{\partial\eta}{\partial t}\right|_z = \frac{\partial}{\partial t}\left(\frac{2z}{L(t)}\right)\Bigg|_z = -\frac{2z\dot{L}(t)}{L^2(t)} = -\frac{\dot{L}(t)}{L(t)}\eta.$$

对 $T(r,z,t) = \hat{T}(\xi,\eta,t)$ 使用链式法则:

$$\left.\frac{\partial T}{\partial t}\right|_{r,z} = \left.\frac{\partial\hat{T}}{\partial t}\right|_{\xi,\eta} + \frac{\partial\hat{T}}{\partial\xi}\left.\frac{\partial\xi}{\partial t}\right|_r + \frac{\partial\hat{T}}{\partial\eta}\left.\frac{\partial\eta}{\partial t}\right|_z = \frac{\partial\hat{T}}{\partial t} - \frac{\dot{R}}{R}\xi\frac{\partial\hat{T}}{\partial\xi} - \frac{\dot{L}}{L}\eta\frac{\partial\hat{T}}{\partial\eta}.$$

因此

$$\frac{\partial T}{\partial t} = \hat{T}_t - \frac{\dot{R}}{R}\xi\hat{T}_\xi - \frac{\dot{L}}{L}\eta\hat{T}_\eta. \tag{18}$$

完全同理,

$$\frac{\partial C}{\partial t} = \hat{C}_t - \frac{\dot{R}}{R}\xi\hat{C}_\xi - \frac{\dot{L}}{L}\eta\hat{C}_\eta.$$

固定 t, z 时,η 与 r 无关, 因此 $\dfrac{\partial\xi}{\partial r} = \dfrac{1}{R(t)}, \dfrac{\partial\eta}{\partial r} = 0$. 由链式法则

$$\frac{\partial T}{\partial r} = \hat{T}_\xi\frac{\partial\xi}{\partial r} + \hat{T}_\eta\frac{\partial\eta}{\partial r} = \frac{1}{R}\hat{T}_\xi, \quad \frac{\partial C}{\partial r} = \frac{1}{R}\hat{C}_\xi. \quad \Longrightarrow \quad \frac{\partial}{\partial r} = \frac{1}{R(t)}\frac{\partial}{\partial\xi}.$$

固定 t, r 时,ξ 与 z 无关, 并且 $\dfrac{\partial\eta}{\partial z} = \dfrac{2}{L(t)}$. 所以

$$\frac{\partial T}{\partial z} = \frac{2}{L}\hat{T}_\eta, \quad \frac{\partial C}{\partial z} = \frac{2}{L}\hat{C}_\eta, \quad \Longrightarrow \quad \frac{\partial}{\partial z} = \frac{2}{L(t)}\frac{\partial}{\partial\eta}.$$

现在我们处理(6)式的变换.
左端为 $\rho(C)c_p(C)T_t$. 由(18)式, 并将 C 写成 $\hat{C}$, 得到

$$\rho(\hat{C})c_p(\hat{C})\left(\hat{T}_t - \frac{\dot{R}}{R}\xi\hat{T}_\xi - \frac{\dot{L}}{L}\eta\hat{T}_\eta\right). \tag{19}$$

观察右端, 有

$$\begin{aligned}\frac{1}{r}\frac{\partial}{\partial r}(rkT_r) =& \frac{1}{R\xi}\cdot\frac{1}{R}\frac{\partial}{\partial\xi}\left[\xi k(\hat{C})\hat{T}_\xi\right] = \frac{1}{R^2}\frac{1}{\xi}\frac{\partial}{\partial\xi}\left[\xi k(\hat{C})\hat{T}_\xi\right].\\ \frac{\partial}{\partial z}(kT_z) =& \frac{2}{L}\frac{\partial}{\partial\eta}\left[k(\hat{C})\frac{2}{L}\hat{T}_\eta\right] = \frac{4}{L^2}\frac{\partial}{\partial\eta}\left[k(\hat{C})\hat{T}_\eta\right].\end{aligned} \tag{20}$$

将式 (19)、(20)式代回(6)式, 得到一般 $R(t), L(t)$ 下的温度方程

$$\rho(\hat{C})c_p(\hat{C})\left[\hat{T}_t - \frac{\dot{R}}{R}\xi\hat{T}_\xi - \frac{\dot{L}}{L}\eta\hat{T}_\eta\right] = \frac{1}{R^2(t)}\frac{1}{\xi}\frac{\partial}{\partial\xi}\left[\xi k(\hat{C})\hat{T}_\xi\right] + \frac{4}{L^2(t)}\frac{\partial}{\partial\eta}\left[k(\hat{C})\hat{T}_\eta\right].$$

若 $R = \gamma R_0, L = \gamma L_0$, 则

$$\frac{\dot{R}}{R} = \frac{\dot{\gamma}}{\gamma}, \quad \frac{\dot{L}}{L} = \frac{\dot{\gamma}}{\gamma}, \quad \frac{1}{R^2} = \frac{1}{\gamma^2 R_0^2}, \quad \frac{4}{L^2} = \frac{4}{\gamma^2 L_0^2}.$$

因此得到等比例收缩后的温度方程

$$\rho(\hat{C})c_p(\hat{C})\left[\hat{T}_t - \frac{\dot{\gamma}}{\gamma}\left(\xi\hat{T}_\xi + \eta\hat{T}_\eta\right)\right] = \frac{1}{\gamma^2 R_0^2}\frac{1}{\xi}\frac{\partial}{\partial\xi}\left[\xi k(\hat{C})\hat{T}_\xi\right] + \frac{4}{\gamma^2 L_0^2}\frac{\partial}{\partial\eta}\left[k(\hat{C})\hat{T}_\eta\right]. \tag{21}$$

现在我们处理(11)式的变换.

左端变换为

$$\rho(\hat{C})\left[\hat{C}_t - \frac{\dot{R}}{R}\xi\hat{C}_\xi - \frac{\dot{L}}{L}\eta\hat{C}_\eta\right].$$

右端变换为

$$\begin{aligned}\frac{1}{r}\frac{\partial}{\partial r}\left(\rho r D C_r\right) &= \frac{1}{R^2}\frac{1}{\xi}\frac{\partial}{\partial\xi}\left[\rho(\hat{C})\xi D(\hat{C},\hat{T})\hat{C}_\xi\right].\\ \frac{\partial}{\partial z}(\rho D C_z) &= \frac{4}{L^2}\frac{\partial}{\partial\eta}\left[\rho(\hat{C})D(\hat{C},\hat{T})\hat{C}_\eta\right].\end{aligned}$$

从而我们得到一般 $R(t), L(t)$ 下的水分方程

$$\rho(\hat{C})\left[\hat{C}_t - \frac{\dot{R}}{R}\xi\hat{C}_\xi - \frac{\dot{L}}{L}\eta\hat{C}_\eta\right] = \frac{1}{R^2(t)}\frac{1}{\xi}\frac{\partial}{\partial\xi}\left[\rho(\hat{C})\xi D(\hat{C},\hat{T})\hat{C}_\xi\right] + \frac{4}{L^2(t)}\frac{\partial}{\partial\eta}\left[\rho(\hat{C})D(\hat{C},\hat{T})\hat{C}_\eta\right].$$

代入 $R = \gamma R_0, L = \gamma L_0$ 后得等比例收缩后的水分方程

$$\begin{aligned}&\rho(\hat{C})\left[\hat{C}_t-\frac{\dot{\gamma}}{\gamma}\left(\xi\hat{C}_\xi+\eta\hat{C}_\eta\right)\right]\\&=\frac{1}{\gamma^2R_0^2}\frac{1}{\xi}\frac{\partial}{\partial\xi}\left[\rho(\hat{C})\xi D(\hat{C},\hat{T})\hat{C}_\xi\right]+\frac{4}{\gamma^2L_0^2}\frac{\partial}{\partial\eta}\left[\rho(\hat{C})D(\hat{C},\hat{T})\hat{C}_\eta\right].\end{aligned}\tag{22}$$

由于坐标变换只改变自变量的表示方式, 不改变经验公式本身. 因此直接作复合代入, 得到

$$\begin{cases}\rho(\hat{C})=760+90\hat{C}, & c_p(\hat{C})=1850+2150\dfrac{\hat{C}}{\hat{C}+1},\\ k(\hat{C})=0.12+0.20\dfrac{\hat{C}}{\hat{C}+1}, & D(\hat{C},\hat{T})=4.2\times10^{-4}\exp\left(-\dfrac{0.30}{\hat{C}}\right)\exp\left(-\dfrac{3850}{\hat{T}}\right).\end{cases}\tag{23}$$

初始条件显然是保持不变的, 即

$$\hat{T}(\xi,\eta,0)=301.15\,\mathrm{K},\quad \hat{C}(\xi,\eta,0)=2.55\,\mathrm{kg\,kg^{-1}}\tag{24}$$

下开始处理边界条件(13)、(14)的变换.

对于 $T_r|_{r=0}=0, C_r|_{r=0}=0$, 使用 $T_r=R^{-1}\hat{T}_\xi$, $C_r=R^{-1}\hat{C}_\xi$, 且 $R(t)>0$ 便得

$$\hat{T}_\xi(0,\eta,t)=0,\qquad \hat{C}_\xi(0,\eta,t)=0.\tag{25}$$

对于径向外表面 $r=R(t)\leftrightsquigarrow\xi=1$ 上的边界条件

$$\begin{aligned}k(C)T_r&=h(T_s-T_a)+h_m(\rho_s-\rho_a)L_v.\\-\rho(C)D(C,T)C_r&=h_m(\rho_s-\rho_a).\end{aligned}$$

分别带入 $T_r=R^{-1}\hat{T}_\xi$ 与 $\hat{T}_s=\hat{T}(1,\eta,t)$, 即得

$$\begin{aligned}-\frac{k(\hat{C})}{R(t)}\hat{T}_\xi\Big|_{\xi=1}&=h(\hat{T}_s-T_a)+h_m(\rho_s-\rho_a)L_v.\\-\frac{\rho(\hat{C})D(\hat{C},\hat{T})}{R(t)}\hat{C}_\xi\Big|_{\xi=1}&=h_m(\rho_s-\rho_a).\end{aligned}\qquad\xi=1$$

等比例收缩时 $R=\gamma R_0$, 因此

$$\begin{aligned} -\frac{k(\hat{C})}{\gamma R_0}\hat{T}_\xi\Big|_{\xi=1} &= h(\hat{T}_s-T_a)+h_m(\rho_s-\rho_a)L_v,\\ -\frac{\rho(\hat{C})D(\hat{C},\hat{T})}{\gamma R_0}\hat{C}_\xi\Big|_{\xi=1} &= h_m(\rho_s-\rho_a). \end{aligned} \tag{26}$$

对于轴向下端面 $z=-L(t)/2 \leftrightsquigarrow \eta=-1$ 的边界条件

$$\begin{aligned} k(C)T_z &= h(T_s-T_a)+h_m(\rho_s-\rho_a)L_v,\\ \rho(C)D(C,T)C_z &= h_m(\rho_s-\rho_a). \end{aligned}$$

代入 $T_z=(2/L)\hat{T}_\eta$, $C_z=(2/L)\hat{C}_\eta$, 并有等比收缩 $L(t)=\gamma(t)L_0$, 即得

$$\begin{aligned} \frac{2k(\hat{C})}{\gamma(t)L_0}\hat{T}_\eta\Big|_{\eta=-1} &= h(\hat{T}_s-T_a)+h_m(\rho_s-\rho_a)L_v,\\ \frac{2\rho(\hat{C})D(\hat{C},\hat{T})}{\gamma(t)L_0}\hat{C}_\eta\Big|_{\eta=-1} &= h_m(\rho_s-\rho_a). \end{aligned} \qquad \eta=-1 \tag{27}$$

上端面式类似的, 变换得

$$\begin{aligned} -\frac{2k(\hat{C})}{\gamma L_0}\hat{T}_\eta &= h(\hat{T}_s-T_a)+h_m(\rho_s-\rho_a)L_v,\\ -\frac{2\rho(\hat{C})D(\hat{C},\hat{T})}{\gamma L_0}\hat{C}_\eta &= h_m(\rho_s-\rho_a). \end{aligned} \qquad \eta=1. \tag{28}$$

至此, 我们得到了适用于问题四的方程组:

温度方程(21) , 水分方程(22), 边界条件(25)、(26)、(27)、(28), 参量(23), 初始条件(24).

给出并通过算法10和11, 结果输出至相应文件, 主要结果在表6中展示 ("-" 表示药材半径已经收缩至该值之内).

Algorithm 10: 考虑尺寸收缩的药材烘干传热传质耦合: 公共初始化

Input: 附件 1 环境数据时间序列 $(t_k^{\text{env}}, T_k^{\text{env}}, C_k^{\text{env}})$
附件 2 收缩时间序列 $(t_k^{\text{shr}}, \gamma_k)$, 初始半径 R_0, 初始柱长 L_0
网格参数 N_r, N_z; 物性函数 $\rho(C), c_p(C), k(C), D(C,T), \rho_{\text{sat}}(T)$
换热传质参数 h, h_m, L_v; 初始条件 T_0, C_0
大气压 P_a, 气体常数 R_a; 水分终止阈值 C_{limit}, 最大模拟时间 $t_{\max}$
Output: 插值函数 $T_\infty(t), C_\infty(t), \gamma(t)$
固定欧拉网格 r_i, z_j, 索引映射 idx(i,j)
初始场 $T_{i,j}^{(0)}$, $C_{i,j}^{(0)}$
读取附件 1, 得到 $t^{\text{env}}, T^{\text{env}}, C^{\text{env}}$
$T^{\text{env}} \leftarrow T^{\text{env}} + 273.15$; // 转为开尔文温度
构造线性插值: $T_\infty(t) \leftarrow \text{Interp1D}(t^{\text{env}}, T^{\text{env}})$
$C_\infty(t) \leftarrow \text{Interp1D}(t^{\text{env}}, C^{\text{env}})$
读取附件 2, 得到 t^{shr}, γ_k
构造线性插值: $\gamma(t) \leftarrow \text{Interp1D}(t^{\text{shr}}, \gamma_k)$; // 半径收缩比 $\gamma(t) = R(t)/R_0$
$\Delta r \leftarrow 0.001$, $\Delta z \leftarrow L_0/N_z$
$r_i \leftarrow (i+0.5)\Delta r,\ i = 0, \ldots, N_r - 1$
$z_j \leftarrow -L_0/2 + (j+0.5)\Delta z,\ j = 0, \ldots, N_z - 1$
定义索引映射 idx$(i,j) \leftarrow i + N_r j$
$T_{i,j}^{(0)} \leftarrow T_0,\ C_{i,j}^{(0)} \leftarrow C_0,\ \forall i,j$
$t \leftarrow 0$, $\Delta t \leftarrow 60$; // 时间步长 60 s
$T_{\text{records}} \leftarrow [\text{Profile}(T^{(0)})]$, $C_{\text{records}} \leftarrow [\text{Profile}(C^{(0)})]$
$next_table_t \leftarrow 0$

表 **6** 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm					
	0	0.5	1	1.5	2	药材表面
6	1.5813	1.3634	0.7361	-	-	0.0283
12	0.5288	0.4451	0.2149	-	-	0.0283
18	0.2742	0.2358	0.1214	-	-	0.0283
24	0.1944	0.1707	0.0963	-	-	0.0283
30	0.1588	0.1413	0.0843	-	-	0.0283
32.283	0.1500	0.1340	0.0813	-	-	0.0283

Algorithm 11: 考虑尺寸收缩的药材烘干传热传质耦合主求解循环

while $t \le t_{\max}$ **do**

  $\gamma \leftarrow \gamma(t)$; $\dot{\gamma} \leftarrow \big(\gamma(t+\Delta t)-\gamma(t)\big)/\Delta t$; $T_\infty \leftarrow T_\infty(t)$; $C_\infty \leftarrow C_\infty(t)$; $\rho_a \leftarrow C_\infty \cdot P_a/(R_a \cdot T_\infty)$; $R_{\text{phys}} \leftarrow \gamma \cdot R_0$; $i_{\text{surf}} \leftarrow \lfloor R_{\text{phys}}/\Delta r \rfloor$; // 当前物理表面网格索引

  /* 更新非线性物性，基于上一步场 $C^{(m-1)}, T^{(m-1)}$ */

  for $j \leftarrow 0$ **to** $N_z - 1$ **do**

    for $i \leftarrow 0$ **to** $N_r - 1$ **do**

      $\rho_{i,j} \leftarrow \rho(C_{i,j}^{(m-1)})$; $cp_{i,j} \leftarrow c_p(C_{i,j}^{(m-1)})$; $k_{i,j} \leftarrow k(C_{i,j}^{(m-1)})$; $D_{i,j} \leftarrow D(C_{i,j}^{(m-1)}, T_{i,j}^{(m-1)})$

  /* 组装温度稀疏线性系统 $A_T T^{n+1} = b_T$ */

  $A_T \leftarrow \text{ZerosSparse}(N,N)$; $b_T \leftarrow \text{Zeros}(N)$ **for** $j \leftarrow 0$ **to** $N_z - 1$ **do**

    for $i \leftarrow 0$ **to** $N_r - 1$ **do**

      $n \leftarrow \text{idx}(i,j)$; $V \leftarrow r_i \Delta r \Delta z$ **if** $i > i_{\text{surf}}$ **then**

        $A_T(n,n) \leftarrow 1$; $b_T(n) \leftarrow T_{i_{\text{surf}},j}^{(m-1)}$; // 外部节点赋值为表面值

        continue

      $r^+ \leftarrow r_i + 0.5\Delta r$; $r^- \leftarrow r_i - 0.5\Delta r$; **if** $i = 0$ **then**

        $r^- \leftarrow 0$

      $A_{r^+} \leftarrow r^+ \Delta z$; $A_{r^-} \leftarrow r^- \Delta z$; $A_z \leftarrow r_i \Delta r$; $a_P \leftarrow \rho_{i,j}\, cp_{i,j}\, V/\Delta t$; $b_T(n) \leftarrow a_P\, T_{i,j}^{(m-1)}$ /* 径向离散；物理表面 $i = i_{\text{surf}}$：对流换热 + 蒸发潜热 */

      if $i == i_{\text{surf}}$ **then**

        $a_P \leftarrow a_P + k_{i,j} A_{r^+}/\Delta r + h A_{r^+}$; evap $\leftarrow h_m\big(\rho_{\text{sat}}(T_{i,j}^{(m-1)}) - \rho_a\big) L_v$;

        $b_T(n) \leftarrow b_T(n) + h A_{r^+} T_\infty - \text{evap}\, A_{r^+}$; 在 A_T 加入与 $T_{i-1,j}$ 的耦合项

      else

        if $i < i_{\text{surf}}$ **then**

          $a_E \leftarrow k_{i,j} A_{r^+}/\Delta r$; $a_P \leftarrow a_P + a_E$; 在 A_T 加入与 $T_{i+1,j}$ 的耦合项

        if $i > 0$ **then**

          $a_W \leftarrow k_{i,j} A_{r^-}/\Delta r$; $a_P \leftarrow a_P + a_W$; 在 A_T 加入与 $T_{i-1,j}$ 的耦合项

      /* 轴向离散；上下端面对流 + 蒸发潜热 */

      if $j = 0$ **or** $j = N_z - 1$ **then**

        $a_P \leftarrow a_P + k_{i,j} A_z/\Delta z + h A_z$; evap $\leftarrow h_m\big(\rho_{\text{sat}}(T_{i,j}^{(m-1)}) - \rho_a\big) L_v$; $b_T(n) \leftarrow b_T(n) + h A_z T_\infty - \text{evap}\, A_z$; 在 A_T 加入与相邻轴向节点的耦合项

      else

        $a_N \leftarrow k_{i,j} A_z/\Delta z$; $a_S \leftarrow k_{i,j} A_z/\Delta z$; $a_P \leftarrow a_P + a_N + a_S$; 在 A_T 加入与 $T_{i,j+1}, T_{i,j-1}$ 的耦合项

      $A_T(n,n) \leftarrow a_P$

  $T^{\text{vec}} \leftarrow \text{SparseSolve}(A_T, b_T)$; $T^{(m)} \leftarrow \text{Reshape}(T^{\text{vec}}, N_r, N_z)$ /* 组装水分浓度稀疏线性系统 $A_C C^{n+1} = b_C$ */

  $A_C \leftarrow \text{ZerosSparse}(N,N)$; $b_C \leftarrow \text{Zeros}(N)$ **for** $j \leftarrow 0$ **to** $N_z - 1$ **do**

    for $i \leftarrow 0$ **to** $N_r - 1$ **do**

      $n \leftarrow \text{idx}(i,j)$; $V \leftarrow r_i \Delta r \Delta z$ **if** $i > i_{\text{surf}}$ **then**

        $A_C(n,n) \leftarrow 1$; $b_C(n) \leftarrow C_{i_{\text{surf}},j}^{(m-1)}$; // 外部节点赋值为表面值

        continue

      $D_{ij} \leftarrow D(C_{i,j}^{(m-1)}, T_{i,j}^{(m)})$; $r^+ \leftarrow r_i + 0.5\Delta r$; $r^- \leftarrow r_i - 0.5\Delta r$; **if** $i = 0$ **then**

        $r^- \leftarrow 0$

      $A_{r^+} \leftarrow r^+ \Delta z$; $A_{r^-} \leftarrow r^- \Delta z$; $A_z \leftarrow r_i \Delta r$; $a_P \leftarrow V/\Delta t$; $b_C(n) \leftarrow a_P\, C_{i,j}^{(m-1)}$ /* 径向离散；物理表面 $i = i_{\text{surf}}$：对流传质边界 */

      if $i == i_{\text{surf}}$ **then**

        $a_P \leftarrow a_P + D_{ij} A_{r^+}/\Delta r + h_m A_{r^+}$; $b_C(n) \leftarrow b_C(n) + h_m A_{r^+}\big(\rho_{\text{sat}}(T_{i,j}^{(m)}) - \rho_a\big)$; 在 A_C 加入与 $C_{i-1,j}$ 的耦合项

      else

        if $i < i_{\text{surf}}$ **then**

          $a_E \leftarrow D_{ij} A_{r^+}/\Delta r$; $a_P \leftarrow a_P + a_E$; 在 A_C 加入与 $C_{i+1,j}$ 的耦合项

        if $i > 0$ **then**

          $a_W \leftarrow D_{ij} A_{r^-}/\Delta r$; $a_P \leftarrow a_P + a_W$; 在 A_C 加入与 $C_{i-1,j}$ 的耦合项

      /* 轴向离散；上下端面对流传质边界 */

      if $j = 0$ **or** $j = N_z - 1$ **then**

        $a_P \leftarrow a_P + D_{ij} A_z/\Delta z + h_m A_z$; $b_C(n) \leftarrow b_C(n) + h_m A_z\big(\rho_{\text{sat}}(T_{i,j}^{(m)}) - \rho_a\big)$; 在 A_C 加入与相邻轴向节点的耦合项

      else

        $a_N \leftarrow D_{ij} A_z/\Delta z$; $a_S \leftarrow D_{ij} A_z/\Delta z$; $a_P \leftarrow a_P + a_N + a_S$; 在 A_C 加入与 $C_{i,j+1}, C_{i,j-1}$ 的耦合项

      $A_C(n,n) \leftarrow a_P$

  $C^{\text{vec}} \leftarrow \text{SparseSolve}(A_C, b_C)$; $C^{(m)} \leftarrow \text{Reshape}(C^{\text{vec}}, N_r, N_z)$; $C^{(m)} \leftarrow \max(C^{(m)}, 10^{-6})$; // 截断负值

  /* 更新时刻、剖面插值记录；超出当前半径填 `NaN/Excel"/"` */

  $t \leftarrow t + \Delta t$; $R_{\text{phys}}^{\text{cm}} = \gamma \cdot R_{0,\text{cm}}$; $i_{\text{surf}} = \lfloor R_{\text{phys}}^{\text{cm}}/(\Delta r \cdot 100) \rfloor$; $C_{\text{surf}} = C^{(m)}(i_{\text{surf}}, N_z/2)$; 插值剖面至固定物理距离 $\{0, 0.1, \ldots, 2.0\}$ cm; 超出当前半径置 NaN(Excel 输出为"/"); 追加记录 $(t, \text{profile}, C_{\text{surf}})$ /* 每 6 小时输出表格行 */

  if $t \ge next_table_t$ **then**

    输出表行: $t/3600$; 径向点位 0, 0.5, 1.0, 1.5, 2.0 cm; 超出输出 —; 输出药材表面值; $next_table_t \leftarrow next_table_t + 6 * 3600$

  /* 烘干合格判据：全部内部节点水分低于阈值 */

  if $\text{all}\big(C^{(m)}(1 : i_{\text{surf}}+1, :) < C_{\text{limit}}\big)$ **then**

    $t_{\text{finish}} \leftarrow t$; **break**

/* 输出结果文件 `result4.xlsx` */

打开 `result4.xlsx`, 写入全部记录; 超出半径单元格填充"/"; 数值单元格格式 `"0.0000"`; 保存文件 **return** t_{finish}

六、 灵敏度分析与误差分析

Algorithm 12: 问题 2 模型求解（SolveModel）

Input: 参数集 P，插值方式 $kind$

Output: 3h 末 (T_c, C_c, C_s)

构造插值函数 f_T, f_C；

初始化网格 N_r, N_z, N，初始场 $T \leftarrow T_0$，$C \leftarrow C_0$；

$t \leftarrow 0$；

while $t < t_{end}$ **do**

  $\Delta t \leftarrow \min(dt, t_{end} - t)$，$t \leftarrow t + \Delta t$；

  $T_\infty \leftarrow f_T(t)$，$C_\infty \leftarrow f_C(t)$，$\rho_a \leftarrow C_\infty P_a/(R_a T_\infty)$；

  组装温度矩阵 (A_T, b_T)，解 $T \leftarrow A_T^{-1} b_T$；

  组装水分矩阵 (A_C, b_C)，解 $C \leftarrow A_C^{-1} b_C$，$C \leftarrow \max(C, 10^{-6})$；

取中截面剖面，提取 T_c, C_c, C_s；

return (T_c, C_c, C_s)

Algorithm 13: 灵敏度分析（SensitivityAnalysis）

Input: 基准参数 P_0，参数集 $\{h, h_m, D_0, C_0\}$，扰动 $\delta \in \{-0.2, -0.1, 0, 0.1, 0.2\}$

Output: 灵敏度结果表

$(T_c^0, C_c^0, C_s^0) \leftarrow \text{SolveModel}(P_0)$；

foreach 参数p **do**

  foreach 扰动δ **do**

    $P \leftarrow P_0.\text{copy}()$，$P.p \leftarrow P_0.p\,(1+\delta)$；

    $(T_c, C_c, C_s) \leftarrow \text{SolveModel}(P)$；

    记录 $(p, \delta, T_c, C_c, C_s)$；

foreach 参数p，扰动δ **do**

  $S_y \leftarrow \dfrac{(y - y_0)/y_0}{\delta}$，　$y \in \{T_c, C_c, C_s\}$；

return 灵敏度表

Algorithm 14: 误差分析（ErrorAnalysis）

Input: 基准参数 P_0，采样数 n，标准差 σ

Output: 收敛性表、Monte Carlo 样本

；　// 数值误差：网格/时间步收敛性

foreach 方案$(dr, dt) \in \{(0.1, 1), (0.05, 1), (0.1, 0.5), (0.05, 0.5)\}$ **do**

  $P \leftarrow P_0.\text{copy}()$，$P.dr \leftarrow dr$，$P.dt \leftarrow dt$；

  $(T_c, C_c, C_s) \leftarrow \text{SolveModel}(P)$；

  计算相对变化，记录；

；　// 模型误差：Monte Carlo 误差传播

for $k = 1$ **to** n **do**

  $P \leftarrow P_0.\text{copy}()$；

  foreach 参数$q \in \{D_0, h_m, h, \rho_0, c_p, k_0\}$ **do**

    $P.q \leftarrow P.q\,(1 + \mathcal{N}(0, \sigma^2))$；

  $(T_c, C_c, C_s) \leftarrow \text{SolveModel}(P)$；

  存入样本；

对每个指标计算均值、标准差、95% 置信区间；

return 收敛性表、MC 样本

Algorithm 15: 问题 2 灵敏度与误差分析主流程

Input: 附件 1，基准参数 P_0

Output: 结果文件、四子图

读取附件 1，构造 f_T, f_C；

$results \leftarrow$ SensitivityAnalysis(P_0)；

$df_{conv} \leftarrow$ ErrorAnalysis 的收敛性部分；

$df_{mc} \leftarrow$ ErrorAnalysis 的 Monte Carlo 部分；

绘制四子图（T_c, C_c, C_s 灵敏度 + 相对灵敏度条形图）；

保存 `sensitivity_q2.xlsx`、`convergence_q2.xlsx`、`montecarlo_q2.xlsx`、`*.png`；

return

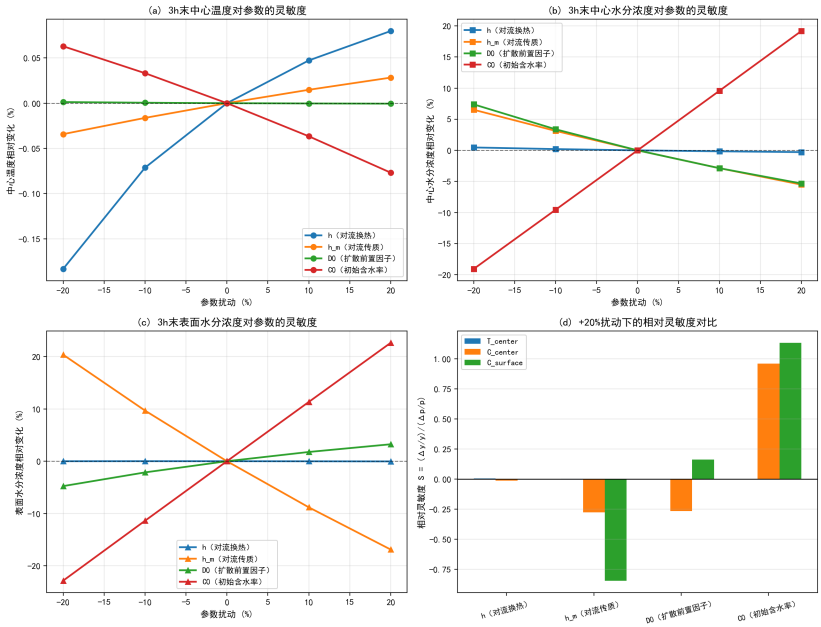


图 **5**　灵敏度分析与误差分析

对问题 2 模型进行单因素灵敏度分析，选取对流换热系数 h、对流传质系数 h_m、扩散系数前置因子 D_0 和初始含水率 C_0 作为扰动对象，扰动幅度取 $\pm 10\%$ 和 $\pm 20\%$. 结果表明:

1. 3h 末中心温度对四个参数的相对灵敏度均小于 0.005，说明预热平衡阶段温度场主要由烘房环境边界条件控制，物性参数影响可忽略;
2. 中心水分浓度对初始含水率 C_0 最敏感 ($|S| \approx 0.96$)，其次为对流传质系数 h_m($|S| \approx 0.28$) 和扩散系数 D_0($|S| \approx 0.27$，对流换热系数 h 影响最弱 ($|S| \approx 0.015$);
3. 表面水分浓度对 C_0($|S| \approx 1.13$) 和 h_m($|S| \approx 0.83$) 最敏感，说明表面蒸发主要受初始含水量和表面传质控制.

上述结论表明，在实际烘干工艺中，应优先控制药材初始含水率的一致性，并合理调控对流传质系数，以保证烘干品质稳定.

七、 总结、改进与推广

7.1 主要结果

本文我们给出了允许参量变换的温度方程(6)以及水分方程(11), 边界条件(13)、(14).

问题一由于是常参量问题, 我们简化方程(6)、(11), 得到简化后的温度方程(7)以及水分方程(12).

问题二、三是变参量问题, 不能再用第一问简化后的方程, 我们采用保留参量的方程(6)、(11), 参量与求解量互相关联, 因此给出了参量组(15).

问题四是收缩边界问题, 我们将其等比投影到固定的一个区域上, 在这个定区域上研究. 由于做了坐标变换, 对应的所有方程与条件都随之改变, 最终得到了温度方程(21), 水分方程(22), 参量(23), 初始条件(24), 边界条件(25)、(26)、(27)、(28).

7.2 内部水汽蒸发的考虑

本文在建立水分扩散方程时, 将水分迁移视为由浓度梯度驱动的扩散过程, 表面蒸发通过边界条件体现. 然而, 在实际烘干过程中, 药材中心区域并非完全不存在水汽蒸发. 当内部温度升高到一定程度时, 中心区域的水分也会在局部孔隙中发生相变, 生成水汽并参与迁移.

本文模型将水汽生成与迁移全部归入等效扩散系数 $D(C,T)$ 中, 属于工程简化处理. 更完善的模型可以引入内部蒸发源项, 将水分扩散方程改写为

$$\frac{\partial C}{\partial t} = \frac{1}{r}\frac{\partial}{\partial r}\left(rD(C,T)\frac{\partial C}{\partial r}\right) + \frac{\partial}{\partial z}\left(D(C,T)\frac{\partial C}{\partial z}\right) + S_{\text{evap}}(r,z,t),$$

其中 S_{evap} 表示单位体积、单位时间内由液态水相变生成的水汽质量, 可由局部温度与饱和水汽压关系确定. 引入该源项后, 传热方程中也需要相应加入相变潜热消耗项, 形成更严格的热湿耦合模型.

7.3 表面水汽饱和假设的改进

本文在计算边界上的水汽浓度 ρ_s 时, 直接假设药材表面处于水汽饱和状态:

$$\rho_s = \frac{P_{\text{sat}}(T_s)}{R_v T_s}.$$

这一假设简化了边界条件, 但过于理想化. 实际烘干过程中, 药材表面的水分活度 a_w 通常小于 1, 且随含水率降低而减小, 表面水汽浓度应写为

$$\rho_s = a_w(C_s)\cdot\frac{P_{\text{sat}}(T_s)}{R_v T_s},$$

其中 $a_w(C_s)$ 与表面含水率 C_s 相关, 可由吸附等温线确定. 常用的经验模型包括 GAB 模型、BET 模型或简化的线性关系. 引入 $a_w(C_s)$ 后, 表面蒸发通量将随含水率降低而减小, 更符合实际干燥后期表面水分难以排出的现象.

7.4 非均匀收缩与热胀冷缩的考虑

本文在问题四中采用等比例均匀收缩假设, 即认为药材各点按相同比例收缩, 径向和轴向收缩比一致:

$$R(t) = \gamma(t)R_0, \qquad L(t) = \gamma(t)L_0.$$

这一假设简化了移动边界的处理, 但与实际物理过程存在偏差. 一方面, 药材内部温度分布不均匀, 表层温度高、中心温度低, 热胀冷缩的程度必然存在空间差异; 另一方面, 水分蒸发主要发生在表层, 表层的失水收缩比中心更剧烈, 导致收缩在空间上不均匀.

更一般的移动边界模型材料点的运动速度不再与半径成正比, 而是与局部温度和含水率相关:

$$v_s(r, z, t) = f(T(r, z, t), C(r, z, t)).$$

此时坐标变换后的控制方程将保留额外的对流项, 不再完全抵消, 方程形式更复杂, 但能更真实地反映非均匀收缩过程. 该模型需要附件 2 之外的更多实验数据支持, 例如不同位置的收缩率或温度分布, 可作为模型推广的进一步方向.

7.5 算法改进

本文第四问采用有限体积法离散控制方程, 时间项全隐式, 物性参数滞后一步. 由于药材尺寸随水分流失持续收缩, 表面节点动态变化, 矩阵组装与求解的计算量较大. 我们可以对时间步长的算法上改进.

固定步长 $\Delta t = 60\text{s}$ 在前期预热阶段精度不足, 后期恒温阶段浪费计算. 引入自适应策略, 根据每步水分相对变化率动态调整:

$$\mathrm{err}^n = \max_{i,j} \frac{|C_{ij}^{n+1} - C_{ij}^n|}{C_{ij}^n},$$

$$\Delta t^{n+1} = \begin{cases} \Delta t^n/2, & \mathrm{err}^n > \varepsilon_{\text{up}}, \\ 1.2\,\Delta t^n, & \mathrm{err}^n < \varepsilon_{\text{low}}, \\ \Delta t^n, & \text{otherwise}, \end{cases}$$

并限定 $\Delta t \in [10\text{s}, 300\text{s}]$. 输出时仍按题目要求的 60s 对齐, 用插值补齐记录点. 该策略在保证精度的同时减少总步数, 尤其适合前期变化剧烈、后期趋于平稳的烘干过程.

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于文献检索、赛题条件与小问关系梳理、候选建模思路讨论、数据预处理方法比较、程序代码调试、模型检验方案讨论、图表优化、摘要提炼、语言润色以及全文格式与一致性检查。AI 提供的内容仅作为辅助参考，相关公式、程序和结果均由参赛队进行人工审查、修改与核验。详细使用情况见支撑材料“AI 工具使用详情.pdf”

参考文献

[1] Bergman T L, Lavine A S, Incropera F P, DeWitt D P. Fundamentals of Heat and Mass Transfer[M]. 8th ed. Hoboken: John Wiley & Sons, 2017.

[2] Crank J. The Mathematics of Diffusion[M]. 2nd ed. Oxford: Clarendon Press, 1975.

[3] Bird R B, Stewart W E, Lightfoot E N. Transport Phenomena[M]. Revised 2nd ed. Hoboken: John Wiley & Sons, 2007.

[4] Mujumdar A S. Handbook of Industrial Drying[M]. 4th ed. Boca Raton: CRC Press, 2014.

[5] Castro A M, Mayorga E Y, Moreno F L. Mathematical modelling of convective drying of fruits: A review[J]. Journal of Food Engineering, 2018, 223: 152–167.

[6] de Oliveira V A B, de Lima W C P B, de Farias Neto S R, de Lima A G B. Heat and Mass Diffusion and Shrinkage in Prolate Spheroidal Bodies Based on Non-Equilibrium Thermodynamics: A Numerical Investigation[J]. Journal of Porous Media, 2011, 14(7): 593–605.

[7] Patankar S V. Numerical Heat Transfer and Fluid Flow[M]. Washington: Hemisphere Publishing Corporation; New York: McGraw-Hill, 1980.

题目文件附录 A　支撑材料的文件列表

文件名	用途
result1.xlsx	问题一答案
result2.xlsx	问题二答案
result3.xlsx	问题三答案
result4.xlsx	问题四答案
第一问耦合.py	问题一
拟合.py	问题一
附件 1.xlsx	题目已知条件
附件 2.xlsx	题目已知条件
水分浓度折线图.pdf	问题一
温度折线图.pdf	问题一
稳态温湿度.py	问题二，问题三
第二问.py	问题二
灵敏度分析.py	灵敏度分析
灵敏度分析图.pdf	灵敏度分析
第三问.py	问题三
第四问.py	问题四

题目文件附录 B　Python 源代码

2.1　问题一拟合曲线源代码

Listing 1: 问题一拟合曲线源代码

```python
import numpy as np

# 1. 定义常数 (Alduchov & Eskridge, 1996)
Mv = 0.018015 # 水的摩尔质量 (kg/mol)
Ru = 8.314472 # 通用气体常数 (J/(mol · K))
A = 6.112     # 饱和水汽压系数 (hPa)
B = 17.62     # 温度系数
C = 243.12    # 温度补偿常数 (°C)

def rho_s_original(Ts):
    """原始标准 Magnus 公式计算饱和水汽密度"""
    Tc = Ts - 273.15
    e_s_hPa = A * np.exp((B * Tc) / (C + Tc))
    e_s_Pa = e_s_hPa * 100
    return (Mv * e_s_Pa) / (Ru * Ts)

# 2. 设定温度范围: 20°C 到 50°C
T_min_C = 20
T_max_C = 50
T_min_K = T_min_C + 273.15 # 293.15 K
T_max_K = T_max_C + 273.15 # 323.15 K

# 生成用于拟合的数据点
Ts_data = np.linspace(T_min_K, T_max_K, 1000)
rho_data = rho_s_original(Ts_data)

# 3. 核心技巧: 温度中心化 + 三次多项式拟合
# 计算温度区间的中点 (35°C = 308.15 K)
T_mid = (T_min_K + T_max_K) / 2

# 将温度平移到 0 附近, 避免大数计算导致的精度丢失
Ts_shifted = Ts_data - T_mid

# 进行三次多项式拟合
coeffs = np.polyfit(Ts_shifted, rho_data, 3)

print(f"温度区间: {T_min_C}°C 到 {T_max_C}°C")
print(f"中心温度 T_mid = {T_mid:.2f} K (即 35°C)")
print("-" * 50)
print("三次多项式拟合系数 (从高次到低次):")
print(f"a3 = {coeffs[0]:.6e}")
```

```
print(f"a2 = {coeffs[1]:.6e}")
print(f"a1 = {coeffs[2]:.6e}")
print(f"a0 = {coeffs[3]:.6e}")
print("-" * 50)
print("最终拟合公式 (x = Ts - 308.15):")
print(f"rho_fit(x) = {coeffs[0]:.6e}*x^3 + {coeffs[1]:.6e}*x^2 + {coeffs[2]:.6e}*x +
    {coeffs[3]:.6e}")

# 4. 误差计算
# 计算拟合值
rho_fit_data = np.polyval(coeffs, Ts_shifted)

# 计算相对误差 (%)
relative_error = np.abs((rho_fit_data - rho_data) / rho_data) * 100

print("-" * 50)
print(f"最大相对误差: {np.max(relative_error):.4f}%")
```

2.2 问题一方程求解源代码

Listing 2: 问题一耦合方程求解源代码

```
import numpy as np
import pandas as pd
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.interpolate import interp1d
from openpyxl import load_workbook

# 1. 读取附件1
df_env = pd.read_excel('附件1.xlsx')
t_env = df_env['时间'].values.astype(float)
T_env = df_env['温度'].values.astype(float) + 273.15
C_env = df_env['水分浓度'].values.astype(float)

T_env_func = interp1d(t_env, T_env, kind='linear',
                      bounds_error=False,
                      fill_value=(T_env[0], T_env[-1]))
C_env_func = interp1d(t_env, C_env, kind='linear',
                      bounds_error=False,
                      fill_value=(C_env[0], C_env[-1]))

# 2. 参数
class Params:
    R = 0.02
    L = 0.25
```

```
    Nr = 40
    Nz = 80

    rho = 820.0
    cp = 2600.0
    k = 0.36
    h = 25.0
    hm = 8e-7
    Lv = 2.26e6

    T0 = 301.15
    C0 = 2.55

    t_end = 1800.0

p = Params()
dr = p.R / p.Nr
dz = p.L / p.Nz
N = p.Nr * p.Nz

rho_d = 820.0 / (1 + 2.55) #  231 kg/m³
rho_air = 1.2              # 空气密度
def idx(i, j):
    return i + p.Nr * j
def D_func(C):
    C = np.maximum(C, 1e-6)
    return 7e-9 * np.exp(-0.89 / C)

# 3. 饱和水汽密度拟合（温度边界用）
A3 = 5.641027e-07
A2 = 4.672843e-05
A1 = 2.057439e-03
A0 = 3.944901e-02
T_MID = 308.15
def rho_sat(Ts):
    x = Ts - T_MID
    return A3*x**3 + A2*x**2 + A1*x + A0
def rho_a_func(Ta, Ca):
    Pa = 101325.0
    Ra = 287.05
    return Ca * Pa / (Ra * Ta)

# 4. 网格
r = (np.arange(p.Nr) + 0.5) * dr
z = -p.L/2 + (np.arange(p.Nz) + 0.5) * dz

# 5. 组装温度矩阵
```

```
def assemble_T(T, T_inf, rho_sat_s, rho_a, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)

    evap_flux = p.hm * (rho_sat_s - rho_a) * p.Lv

    for j in range(p.Nz):
        for i in range(p.Nr):
            n = idx(i, j)
            V = r[i] * dr * dz

            r_plus = r[i] + 0.5 * dr
            r_minus = r[i] - 0.5 * dr
            if i == 0:
                r_minus = 0.0

            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

            aP = p.rho * p.cp * V / dt_local
            b[n] = aP * T[i, j]

            if i == p.Nr - 1:
                coeff = p.k * A_rp / dr
                aP += coeff + p.h * A_rp
                b[n] += p.h * A_rp * T_inf - evap_flux * A_rp
                rows.append(n); cols.append(idx(i-1, j)); vals.append(-coeff)
            else:
                if i < p.Nr - 1:
                    aE = p.k * A_rp / dr
                    rows.append(n); cols.append(idx(i+1, j)); vals.append(-aE)
                    aP += aE
                if i > 0:
                    aW = p.k * A_rm / dr
                    rows.append(n); cols.append(idx(i-1, j)); vals.append(-aW)
                    aP += aW

            if j == 0:
                coeff = p.k * A_z / dz
                aP += coeff + p.h * A_z
                b[n] += p.h * A_z * T_inf - evap_flux * A_z
                rows.append(n); cols.append(idx(i, j+1)); vals.append(-coeff)
            elif j == p.Nz - 1:
                coeff = p.k * A_z / dz
                aP += coeff + p.h * A_z
                b[n] += p.h * A_z * T_inf - evap_flux * A_z
```

```
            rows.append(n); cols.append(idx(i, j-1)); vals.append(-coeff)
        else:
            aN = p.k * A_z / dz
            rows.append(n); cols.append(idx(i, j+1)); vals.append(-aN)
            aP += aN
            aS = p.k * A_z / dz
            rows.append(n); cols.append(idx(i, j-1)); vals.append(-aS)
            aP += aS

        rows.append(n); cols.append(n); vals.append(aP)

    A = sp.csr_matrix((vals, (rows, cols)), shape=(N, N))
    return A, b

# 6. 组装水分矩阵
def assemble_C(C, C_inf, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)

    rho_a = rho_air * C_inf

    for j in range(p.Nz):
        for i in range(p.Nr):
            n = idx(i, j)
            V = r[i] * dr * dz
            D = D_func(C[i, j])

            r_plus = r[i] + 0.5 * dr
            r_minus = r[i] - 0.5 * dr
            if i == 0:
                r_minus = 0.0

            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

            aP = V / dt_local
            b[n] = aP * C[i, j]

            # 径向
            if i == p.Nr - 1:
                D_s = D_func(C[i, j])
                coeff = D_s * A_rp / dr
                aP += coeff + p.hm * A_rp
                b[n] += p.hm * A_rp * (rho_a / rho_d)
                rows.append(n); cols.append(idx(i-1, j)); vals.append(-coeff)
            else:
```

```
            if i < p.Nr - 1:
                aE = D * A_rp / dr
                rows.append(n); cols.append(idx(i+1, j)); vals.append(-aE)
                aP += aE
            if i > 0:
                aW = D * A_rm / dr
                rows.append(n); cols.append(idx(i-1, j)); vals.append(-aW)
                aP += aW

        # 轴向
        if j == 0:
            D_s = D_func(C[i, j])
            coeff = D_s * A_z / dz
            aP += coeff + p.hm * A_z
            b[n] += p.hm * A_z * (rho_a / rho_d)
            rows.append(n); cols.append(idx(i, j+1)); vals.append(-coeff)
        elif j == p.Nz - 1:
            D_s = D_func(C[i, j])
            coeff = D_s * A_z / dz
            aP += coeff + p.hm * A_z
            b[n] += p.hm * A_z * (rho_a / rho_d)
            rows.append(n); cols.append(idx(i, j-1)); vals.append(-coeff)
        else:
            aN = D * A_z / dz
            rows.append(n); cols.append(idx(i, j+1)); vals.append(-aN)
            aP += aN
            aS = D * A_z / dz
            rows.append(n); cols.append(idx(i, j-1)); vals.append(-aS)
            aP += aS

        rows.append(n); cols.append(n); vals.append(aP)

    A = sp.csr_matrix((vals, (rows, cols)), shape=(N, N))
    return A, b

# 7. 求解
def solve_all():
    T = np.full((p.Nr, p.Nz), p.T0)
    C = np.full((p.Nr, p.Nz), p.C0)

    times = [0.0]
    T_fields = [T.copy()]
    C_fields = [C.copy()]

    t = 0.0

    # 第一阶段: 0~1 s, 步长 0.1 s
```

```
    print("第一阶段: 0~1 s")
    while t < 1.0 - 1e-9:
        dt_local = min(0.1, 1.0 - t)
        t += dt_local
        T_inf = float(T_env_func(t))
        C_inf = float(C_env_func(t))

        T_surf = T[-1, 0]
        rho_sat_s = rho_sat(T_surf)
        rho_a = rho_a_func(T_inf, C_inf)

        A_T, b_T = assemble_T(T, T_inf, rho_sat_s, rho_a, dt_local)
        T = spla.spsolve(A_T, b_T).reshape((p.Nr, p.Nz))

        A_C, b_C = assemble_C(C, C_inf, dt_local)
        C = spla.spsolve(A_C, b_C).reshape((p.Nr, p.Nz))
        C = np.maximum(C, 1e-6)
        C[0, :] = C[1, :]  # 轴对称条件: 中心 = 相邻节点

        times.append(t)
        T_fields.append(T.copy())
        C_fields.append(C.copy())

    # 第二阶段: 1~1800 s, 步长 1 s
    print("第二阶段: 1~1800 s")
    step = 0
    while t <= p.t_end + 1e-9:
        dt_local = min(1.0, p.t_end - t)
        if dt_local <= 0:
            break
        t += dt_local
        T_inf = float(T_env_func(t))
        C_inf = float(C_env_func(t))

        T_surf = T[-1, 0]
        rho_sat_s = rho_sat(T_surf)
        rho_a = rho_a_func(T_inf, C_inf)

        A_T, b_T = assemble_T(T, T_inf, rho_sat_s, rho_a, dt_local)
        T = spla.spsolve(A_T, b_T).reshape((p.Nr, p.Nz))

        A_C, b_C = assemble_C(C, C_inf, dt_local)
        C = spla.spsolve(A_C, b_C).reshape((p.Nr, p.Nz))
        C = np.maximum(C, 1e-6)
        C[0, :] = C[1, :]  # 轴对称条件: 中心 = 相邻节点

        times.append(t)
```

```
        T_fields.append(T.copy())
        C_fields.append(C.copy())

        step += 1
        if step % 300 == 0:
            print(f" step={step}, t={t:.0f}s, "
                  f"T_c={T[0,0]-273.15:.2f}°C, "
                  f"C_c={C[0,0]:.4f}, C_s={C[-1,0]:.4f}")

    return np.array(times), np.array(T_fields), np.array(C_fields)

print("正在求解...")
times, T_fields, C_fields = solve_all()
print("求解完成！\n")

# 8. 固定 z = 0
Z_FIXED = 0.0
j_fixed = np.argmin(np.abs(z - Z_FIXED))
T_slice = T_fields[:, :, j_fixed] # (nt, Nr)
C_slice = C_fields[:, :, j_fixed]

# 9. 构造输出径向网格（包含中心 r=0）
r_nodes_cm = r * 100.0
r_target_cm = [0, 0.5, 1.0, 1.5, 2.0]
idx_target = [np.argmin(np.abs(r_nodes_cm - rc)) for rc in r_target_cm]
idx_target[0] = 0

t_target = [0, 100, 300, 600, 900, 1200, 1500, 1800]

T_show = np.zeros((len(t_target), len(r_target_cm)))
C_show = np.zeros((len(t_target), len(r_target_cm)))

for iq, i_node in enumerate(idx_target):
    T_r = T_slice[:, i_node]
    C_r = C_slice[:, i_node]
    T_show[:, iq] = np.interp(t_target, times, T_r) - 273.15
    C_show[:, iq] = np.interp(t_target, times, C_r)

T_show[0, :] = p.T0 - 273.15
C_show[0, :] = p.C0

# 10. 终端打印（表1温度3位小数，表2水分浓度4位小数）
print("表 1 30 分钟内药材的温度 (°C)")
header = f"{'时间/s':>8} | " + " | ".join(f"{x:>8}" for x in r_target_cm)
print(header)
print("-" * len(header))
for i, t_val in enumerate(t_target):
```

```
    row = f"{t_val:>8} | " + " | ".join(
        f"{T_show[i, j]:>8.3f}" for j in range(len(r_target_cm))
    )
    print(row)

print("\n表 2 30 分钟内药材的水分浓度 (kg/kg)")
header = f"{'时间/s':>8} | " + " | ".join(f"{x:>8}" for x in r_target_cm)
print(header)
print("-" * len(header))
for i, t_val in enumerate(t_target):
    row = f"{t_val:>8} | " + " | ".join(
        f"{C_show[i, j]:>8.4f}" for j in range(len(r_target_cm))
    )
    print(row)

# 11. 写入 result1.xlsx
r_cm = [round(0.1*k, 1) for k in range(21)]
t_write = np.arange(0.0, 1801.0, 1.0)

idx_write = [np.argmin(np.abs(r_nodes_cm - rc)) for rc in r_cm]
idx_write[0] = 0

T_out = np.zeros((len(t_write), 21))
C_out = np.zeros((len(t_write), 21))
for iq, i_node in enumerate(idx_write):
    T_r = T_slice[:, i_node]
    C_r = C_slice[:, i_node]
    T_out[:, iq] = np.interp(t_write, times, T_r) - 273.15
    C_out[:, iq] = np.interp(t_write, times, C_r)

T_out[0, :] = p.T0 - 273.15
C_out[0, :] = p.C0

wb = load_workbook('result1.xlsx')

# 温度表: 保留 3 位小数
ws = wb['温度']
ws.cell(1, 1, "时间")
for j, rc in enumerate(r_cm):
    ws.cell(1, 2+j, rc)
for i, t in enumerate(t_write):
    ws.cell(2+i, 1, int(t))
    for j in range(21):
        cell = ws.cell(2+i, 2+j)
        cell.value = round(float(T_out[i, j]), 3) # 数值保留 3 位
        cell.number_format = '0.000'          # 强制显示 3 位小数

```

```
# 水分浓度表: 保留 4 位小数
ws = wb['水分浓度']
ws.cell(1, 1, "时间")
for j, rc in enumerate(r_cm):
    ws.cell(1, 2+j, rc)
for i, t in enumerate(t_write):
    ws.cell(2+i, 1, int(t))
    for j in range(21):
        cell = ws.cell(2+i, 2+j)
        cell.value = round(float(C_out[i, j]), 4) # 数值保留 4 位
        cell.number_format = '0.0000'         # 强制显示 4 位小数

wb.save('result1.xlsx')
print("已写入 result1.xlsx")
```

2.3 问题二稳态环境求解源代码

Listing 3: 问题一拟合曲线源代码

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_excel('附件1.xlsx', sheet_name='Sheet1')

time = df.iloc[:, 0]  # 时间
temp = df.iloc[:, 1]  # 温度
moist = df.iloc[:, 2] # 水分浓度

# 计算稳态值（取最后 1 小时，即最后 1000 s）
mask = time >= (time.max() - 1000)
steady_temp = temp[mask].mean()
steady_moist = moist[mask].mean()

print(f"稳态温度: {steady_temp:.3f} °C")
print(f"稳态水分浓度: {steady_moist:.5f}")

# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False

# 图1: 温度-时间
plt.figure(figsize=(10, 5))
plt.plot(time, temp, color='#d62728', linewidth=1.2, label='温度')
plt.axhline(steady_temp, color='gray', linestyle='--', linewidth=1,
            label=f'稳态温度  {steady_temp:.2f} °C')
plt.xlabel('时间 (s)')
```

```
plt.ylabel('温度 (°C)')
plt.title('温度随时间变化')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('温度折线图.pdf', dpi=300)
plt.show()

# 图2: 水分浓度-时间
plt.figure(figsize=(10, 5))
plt.plot(time, moist, color='#1f77b4', linewidth=1.2, label='水分浓度')
plt.axhline(steady_moist, color='gray', linestyle='--', linewidth=1,
           label=f'稳态水分浓度  {steady_moist:.5f}')
plt.xlabel('时间 (s)')
plt.ylabel('水分浓度')
plt.title('水分浓度随时间变化')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('水分浓度折线图.pdf', dpi=300)
plt.show()
```

2.4 问题二方程求解源代码

Listing 4: 问题一耦合方程求解源代码

```
from pathlib import Path
import numpy as np
import pandas as pd
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.interpolate import interp1d
from openpyxl import load_workbook

# 1. 读取附件1
BASE_DIR = Path(__file__).resolve().parent
df_env = pd.read_excel(BASE_DIR / '附件1.xlsx')

t_env = df_env['时间'].values.astype(float)
T_env = df_env['温度'].values.astype(float) + 273.15
C_env = df_env['水分浓度'].values.astype(float)

T_env_func = interp1d(t_env, T_env, kind='linear',
                      bounds_error=False, fill_value=(T_env[0], T_env[-1]))
C_env_func = interp1d(t_env, C_env, kind='linear',
                      bounds_error=False, fill_value=(C_env[0], C_env[-1]))
```

```
21
22  # 2. 参数
23  class Params:
24      R = 0.02        # 半径 2 cm
25      L = 0.25        # 高度 25 cm
26      dr = 0.001      # 径向步长 0.1 cm
27      Nz = 10
28      h = 25.0
29      hm = 8.0e-7
30      Lv = 2.26e6
31      T0 = 301.15
32      C0 = 2.55
33      t_end = 3 * 3600.0
34
35
36  p = Params()
37  p.Nr = int(round(p.R / p.dr))
38  dr = p.dr
39  dz = p.L / p.Nz
40  N = p.Nr * p.Nz
41
42  Pa = 101325.0
43  Ra = 287.05
44
45
46  def idx(i, j):
47      return i + p.Nr * j
48
49  # 3. 物性
50  def rho_fun(C):
51      return 650.0 + 128.0 * C
52
53  def cp_fun(C):
54      return 1450.0 + 2736.0 * C / (C + 1.0)
55
56  def k_fun(C):
57      return 0.21 + 0.38 * C / (C + 1.0)
58
59  def D_fun(C, T):
60      C_safe = np.maximum(C, 1e-6)
61      return 2.4e-3 * np.exp(-0.45 / C_safe) * np.exp(-3850.0 / T)
62
63  # 4. 饱和水汽密度
64  A3, A2, A1, A0 = 5.641027e-07, 4.672843e-05, 2.057439e-03, 3.944901e-02
65  T_MID = 308.15
66
67  def rho_sat(Ts):
```

```
    Ts_clip = np.clip(Ts, 273.15, 323.15)
    x = Ts_clip - T_MID
    return np.maximum(A3 * x**3 + A2 * x**2 + A1 * x + A0, 0.0)

def rho_a_func(Ta, Ca):
    return Ca * Pa / (Ra * Ta)

# 5. 网格
r = (np.arange(p.Nr) + 0.5) * dr
z = -p.L / 2 + (np.arange(p.Nz) + 0.5) * dz

# 6. 温度矩阵
def assemble_T(T, T_inf, C_field, rho_a, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)

    for j in range(p.Nz):
        for i in range(p.Nr):
            n = idx(i, j)
            V = r[i] * dr * dz
            r_plus = r[i] + 0.5 * dr
            r_minus = r[i] - 0.5 * dr if i > 0 else 0.0
            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

            rho = rho_fun(C_field[i, j])
            cp = cp_fun(C_field[i, j])
            k = k_fun(C_field[i, j])

            aP = rho * cp * V / dt_local
            b[n] = aP * T[i, j]

            if i == p.Nr - 1:
                aW = k * A_rm / dr
                aP += aW
                rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)

                evap = p.hm * (rho_sat(T[i, j]) - rho_a) * p.Lv
                aP += p.h * A_rp
                b[n] += p.h * A_rp * T_inf - evap * A_rp
            else:
                if i < p.Nr - 1:
                    aE = k * A_rp / dr
                    rows.append(n); cols.append(idx(i + 1, j)); vals.append(-aE)
                    aP += aE
                if i > 0:
```

```
                aW = k * A_rm / dr
                rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)
                aP += aW

        if j == 0:
            coeff = k * A_z / dz
            aP += coeff
            rows.append(n); cols.append(idx(i, j + 1)); vals.append(-coeff)

            evap = p.hm * (rho_sat(T[i, j]) - rho_a) * p.Lv
            aP += p.h * A_z
            b[n] += p.h * A_z * T_inf - evap * A_z
        elif j == p.Nz - 1:
            coeff = k * A_z / dz
            aP += coeff
            rows.append(n); cols.append(idx(i, j - 1)); vals.append(-coeff)

            evap = p.hm * (rho_sat(T[i, j]) - rho_a) * p.Lv
            aP += p.h * A_z
            b[n] += p.h * A_z * T_inf - evap * A_z
        else:
            aN = k * A_z / dz
            aS = k * A_z / dz
            rows.append(n); cols.append(idx(i, j + 1)); vals.append(-aN)
            rows.append(n); cols.append(idx(i, j - 1)); vals.append(-aS)
            aP += aN + aS

        rows.append(n); cols.append(n); vals.append(aP)

    return sp.csr_matrix((vals, (rows, cols)), shape=(N, N)), b

# 7. 水分矩阵
def assemble_C(C, C_inf, T_field, T_inf, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    rho_a = rho_a_func(T_inf, C_inf)

    for j in range(p.Nz):
        for i in range(p.Nr):
            n = idx(i, j)
            V = r[i] * dr * dz
            D = D_fun(C[i, j], T_field[i, j])

            r_plus = r[i] + 0.5 * dr
            r_minus = r[i] - 0.5 * dr if i > 0 else 0.0
            A_rp = r_plus * dz
            A_rm = r_minus * dz
```

```
            A_z = r[i] * dr

            aP = V / dt_local
            b[n] = aP * C[i, j]

            if i == p.Nr - 1:
                aW = D * A_rm / dr
                aP += aW
                rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)

                aP += p.hm * A_rp
                b[n] += p.hm * A_rp * (rho_sat(T_field[i, j]) - rho_a)
            else:
                if i < p.Nr - 1:
                    aE = D * A_rp / dr
                    rows.append(n); cols.append(idx(i + 1, j)); vals.append(-aE)
                    aP += aE
                if i > 0:
                    aW = D * A_rm / dr
                    rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)
                    aP += aW

            if j == 0:
                coeff = D * A_z / dz
                aP += coeff
                rows.append(n); cols.append(idx(i, j + 1)); vals.append(-coeff)

                aP += p.hm * A_z
                b[n] += p.hm * A_z * (rho_sat(T_field[i, j]) - rho_a)
            elif j == p.Nz - 1:
                coeff = D * A_z / dz
                aP += coeff
                rows.append(n); cols.append(idx(i, j - 1)); vals.append(-coeff)

                aP += p.hm * A_z
                b[n] += p.hm * A_z * (rho_sat(T_field[i, j]) - rho_a)
            else:
                aN = D * A_z / dz
                aS = D * A_z / dz
                rows.append(n); cols.append(idx(i, j + 1)); vals.append(-aN)
                rows.append(n); cols.append(idx(i, j - 1)); vals.append(-aS)
                aP += aN + aS

            rows.append(n); cols.append(n); vals.append(aP)

    return sp.csr_matrix((vals, (rows, cols)), shape=(N, N)), b
```

```
# 8. 径向剖面函数
def radial_profile_at_mid_z_full(field):
    """用于写 result2: 插值到 0,0.1,...,2.0 cm"""
    if p.Nz % 2 == 1:
        prof = field[:, p.Nz // 2]
    else:
        j1, j2 = p.Nz // 2 - 1, p.Nz // 2
        prof = 0.5 * (field[:, j1] + field[:, j2])

    r_cm = r * 100.0
    target_cm = np.arange(0.0, 2.0 + 1e-12, 0.1)
    f_interp = interp1d(r_cm, prof, kind='linear',
                        bounds_error=False, fill_value=(prof[0], prof[-1]))
    return f_interp(target_cm)

def radial_profile_at_mid_z_points(field, target_cm):
    """用于运行框打印: 插值到指定距离点"""
    if p.Nz % 2 == 1:
        prof = field[:, p.Nz // 2]
    else:
        j1, j2 = p.Nz // 2 - 1, p.Nz // 2
        prof = 0.5 * (field[:, j1] + field[:, j2])

    r_cm = r * 100.0
    f_interp = interp1d(r_cm, prof, kind='linear',
                        bounds_error=False, fill_value=(prof[0], prof[-1]))
    return f_interp(target_cm)

# 9. 求解
#   result2: 0~10800 s, 每秒一行
#   运行框: 只打印 0.5,1.0,...,3.0 h 和 0,0.5,1.0,1.5,2.0 cm
T = np.full((p.Nr, p.Nz), p.T0, dtype=float)
C = np.full((p.Nr, p.Nz), p.C0, dtype=float)

dt_local = 1.0
t = 0.0

# result2 记录
T_records = [radial_profile_at_mid_z_full(T) - 273.15]
C_records = [radial_profile_at_mid_z_full(C)]

# 运行框打印设置
print_hours = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
print_seconds = [h * 3600.0 for h in print_hours]
print_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0])

print("\n表3 3小时内药材的温度（°C）")
```

```
print("时间/h | " + " | ".join(f"{x:.1f}cm" for x in print_cm))
print("-" * 60)

next_print_idx = 0
next_record_idx = 1
C_print_rows = []

while t < p.t_end - 1e-9:
    dt_step = min(dt_local, p.t_end - t)
    t += dt_step

    T_inf = float(T_env_func(t))
    C_inf = float(C_env_func(t))
    rho_a = rho_a_func(T_inf, C_inf)

    A_T, b_T = assemble_T(T, T_inf, C, rho_a, dt_step)
    T = spla.spsolve(A_T, b_T).reshape((p.Nr, p.Nz), order='F')

    A_C, b_C = assemble_C(C, C_inf, T, T_inf, dt_step)
    C = spla.spsolve(A_C, b_C).reshape((p.Nr, p.Nz), order='F')
    C = np.maximum(C, 1e-6)

    # 每秒写入 result2
    if next_record_idx < 10801 and t >= next_record_idx - 1e-9:
        T_records.append(radial_profile_at_mid_z_full(T) - 273.15)
        C_records.append(radial_profile_at_mid_z_full(C))
        next_record_idx += 1

    # 只在指定小时打印表3、表4
    if next_print_idx < len(print_seconds) and t >= print_seconds[next_print_idx] - 1e-9:
        T_prof = radial_profile_at_mid_z_points(T, print_cm) - 273.15
        C_prof = radial_profile_at_mid_z_points(C, print_cm)

        print(f"{t / 3600:>6.1f} | " + " | ".join(f"{v:>8.3f}" for v in T_prof))
        C_print_rows.append((t / 3600, C_prof))

        next_print_idx += 1

print("\n表4 3小时内药材的水分浓度")
print("时间/h | " + " | ".join(f"{x:.1f}cm" for x in print_cm))
print("-" * 60)
for h, prof in C_print_rows:
    print(f"{h:>6.1f} | " + " | ".join(f"{v:>8.4f}" for v in prof))

print('\n计算完成！')

# 10. 写入 result2.xlsx
```

```
#     温度: 3 位小数
#     水分浓度: 4 位小数
#     时间列: 整数
columns = ['时间\\到药材中心的距离'] + [f'{x:.1f}' for x in np.arange(0.0, 2.0 + 1e-12, 0.1)]
time_col = np.arange(0, 10801, 1, dtype=int)

df_T = pd.DataFrame(T_records, columns=columns[1:])
df_T.insert(0, columns[0], time_col)

df_C = pd.DataFrame(C_records, columns=columns[1:])
df_C.insert(0, columns[0], time_col)

df_T = df_T.round(3)
df_C = df_C.round(4)

out_path = BASE_DIR / 'result2.xlsx'
with pd.ExcelWriter(out_path, engine='openpyxl') as writer:
    df_T.to_excel(writer, sheet_name='温度', index=False)
    df_C.to_excel(writer, sheet_name='水分浓度', index=False)

# 强制 Excel 显示格式
wb = load_workbook(out_path)

# 温度 sheet: 数据列保留 3 位小数
ws_T = wb['温度']
for row in ws_T.iter_rows(min_row=2, min_col=2):
    for cell in row:
        if isinstance(cell.value, (int, float)):
            cell.number_format = '0.000'

# 水分浓度 sheet: 数据列保留 4 位小数
ws_C = wb['水分浓度']
for row in ws_C.iter_rows(min_row=2, min_col=2):
    for cell in row:
        if isinstance(cell.value, (int, float)):
            cell.number_format = '0.0000'

wb.save(out_path)

print(f'已写入: {out_path}')
print(f'温度记录行数: {len(T_records)}')
print(f'水分浓度记录行数: {len(C_records)}')
```

2.5 问题三方程求解源代码

Listing 5: 问题一耦合方程求解源代码

```python
from pathlib import Path
import numpy as np
import pandas as pd
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.interpolate import interp1d
from openpyxl import load_workbook

# 1. 读取 result2.xlsx 中 10800 s 的数据
BASE_DIR = Path(__file__).resolve().parent
result2_path = BASE_DIR / 'result2.xlsx'

df_T_all = pd.read_excel(result2_path, sheet_name='温度')
df_C_all = pd.read_excel(result2_path, sheet_name='水分浓度')

row_T = df_T_all[df_T_all.iloc[:, 0] == 10800]
row_C = df_C_all[df_C_all.iloc[:, 0] == 10800]

if row_T.empty or row_C.empty:
    raise ValueError('result2.xlsx 中没有找到 10800 s 的数据行。')

T_profile_cm = row_T.iloc[0, 1:].values.astype(float) # °C
C_profile_cm = row_C.iloc[0, 1:].values.astype(float)

target_cm = np.array([float(c) for c in df_T_all.columns[1:]])


# 2. 参数
class Params:
    R = 0.02
    L = 0.25
    dr = 0.001
    Nz = 10
    h = 25.0
    hm = 8.0e-7
    Lv = 2.26e6
    T0 = 301.15
    C0 = 2.55
    t_start = 10800.0
    t_max = 10 * 24 * 3600.0  # 最多算 10 天，防止死循环


p = Params()
p.Nr = int(round(p.R / p.dr))
dr = p.dr
```

```
dz = p.L / p.Nz
N = p.Nr * p.Nz

Pa = 101325.0
Ra = 287.05

T_STABLE = 50.0 + 273.15
C_STABLE = 0.04997
C_LIMIT = 0.15


def idx(i, j):
    return i + p.Nr * j

# 3. 物性
def rho_fun(C):
    return 650.0 + 128.0 * C

def cp_fun(C):
    return 1450.0 + 2736.0 * C / (C + 1.0)

def k_fun(C):
    return 0.21 + 0.38 * C / (C + 1.0)

def D_fun(C, T):
    C_safe = np.maximum(C, 1e-6)
    return 2.4e-3 * np.exp(-0.45 / C_safe) * np.exp(-3850.0 / T)


# 4. 饱和水汽密度
A3, A2, A1, A0 = 5.641027e-07, 4.672843e-05, 2.057439e-03, 3.944901e-02
T_MID = 308.15


def rho_sat(Ts):
    Ts_clip = np.clip(Ts, 273.15, 323.15)
    x = Ts_clip - T_MID
    return np.maximum(A3 * x**3 + A2 * x**2 + A1 * x + A0, 0.0)

def rho_a_func(Ta, Ca):
    return Ca * Pa / (Ra * Ta)

# 5. 网格
r = (np.arange(p.Nr) + 0.5) * dr
r_cm = r * 100.0
z = -p.L / 2 + (np.arange(p.Nz) + 0.5) * dz

```

```
# 6. 从 result2 恢复二维初始场
f_T = interp1d(target_cm, T_profile_cm, kind='linear',
               bounds_error=False, fill_value=(T_profile_cm[0], T_profile_cm[-1]))
f_C = interp1d(target_cm, C_profile_cm, kind='linear',
               bounds_error=False, fill_value=(C_profile_cm[0], C_profile_cm[-1]))

T_r = f_T(r_cm) + 273.15
C_r = f_C(r_cm)

T = np.tile(T_r[:, None], (1, p.Nz))
C = np.tile(C_r[:, None], (1, p.Nz))
C = np.maximum(C, 1e-6)

# 7. 温度矩阵
def assemble_T(T, T_inf, C_field, rho_a, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)

    for j in range(p.Nz):
        for i in range(p.Nr):
            n = idx(i, j)
            V = r[i] * dr * dz
            r_plus = r[i] + 0.5 * dr
            r_minus = r[i] - 0.5 * dr if i > 0 else 0.0
            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

            rho = rho_fun(C_field[i, j])
            cp = cp_fun(C_field[i, j])
            k = k_fun(C_field[i, j])

            aP = rho * cp * V / dt_local
            b[n] = aP * T[i, j]

            if i == p.Nr - 1:
                aW = k * A_rm / dr
                aP += aW
                rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)

                evap = p.hm * (rho_sat(T[i, j]) - rho_a) * p.Lv
                aP += p.h * A_rp
                b[n] += p.h * A_rp * T_inf - evap * A_rp
            else:
                if i < p.Nr - 1:
                    aE = k * A_rp / dr
                    rows.append(n); cols.append(idx(i + 1, j)); vals.append(-aE)
```

```
                aP += aE
            if i > 0:
                aW = k * A_rm / dr
                rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)
                aP += aW

        if j == 0:
            coeff = k * A_z / dz
            aP += coeff
            rows.append(n); cols.append(idx(i, j + 1)); vals.append(-coeff)

            evap = p.hm * (rho_sat(T[i, j]) - rho_a) * p.Lv
            aP += p.h * A_z
            b[n] += p.h * A_z * T_inf - evap * A_z
        elif j == p.Nz - 1:
            coeff = k * A_z / dz
            aP += coeff
            rows.append(n); cols.append(idx(i, j - 1)); vals.append(-coeff)

            evap = p.hm * (rho_sat(T[i, j]) - rho_a) * p.Lv
            aP += p.h * A_z
            b[n] += p.h * A_z * T_inf - evap * A_z
        else:
            aN = k * A_z / dz
            aS = k * A_z / dz
            rows.append(n); cols.append(idx(i, j + 1)); vals.append(-aN)
            rows.append(n); cols.append(idx(i, j - 1)); vals.append(-aS)
            aP += aN + aS

        rows.append(n); cols.append(n); vals.append(aP)

    return sp.csr_matrix((vals, (rows, cols)), shape=(N, N)), b

# 8. 水分矩阵
def assemble_C(C, C_inf, T_field, T_inf, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    rho_a = rho_a_func(T_inf, C_inf)

    for j in range(p.Nz):
        for i in range(p.Nr):
            n = idx(i, j)
            V = r[i] * dr * dz
            D = D_fun(C[i, j], T_field[i, j])

            r_plus = r[i] + 0.5 * dr
            r_minus = r[i] - 0.5 * dr if i > 0 else 0.0
```

```
            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

            aP = V / dt_local
            b[n] = aP * C[i, j]

            if i == p.Nr - 1:
                aW = D * A_rm / dr
                aP += aW
                rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)

                aP += p.hm * A_rp
                b[n] += p.hm * A_rp * (rho_sat(T_field[i, j]) - rho_a)
            else:
                if i < p.Nr - 1:
                    aE = D * A_rp / dr
                    rows.append(n); cols.append(idx(i + 1, j)); vals.append(-aE)
                    aP += aE
                if i > 0:
                    aW = D * A_rm / dr
                    rows.append(n); cols.append(idx(i - 1, j)); vals.append(-aW)
                    aP += aW

            if j == 0:
                coeff = D * A_z / dz
                aP += coeff
                rows.append(n); cols.append(idx(i, j + 1)); vals.append(-coeff)

                aP += p.hm * A_z
                b[n] += p.hm * A_z * (rho_sat(T_field[i, j]) - rho_a)
            elif j == p.Nz - 1:
                coeff = D * A_z / dz
                aP += coeff
                rows.append(n); cols.append(idx(i, j - 1)); vals.append(-coeff)

                aP += p.hm * A_z
                b[n] += p.hm * A_z * (rho_sat(T_field[i, j]) - rho_a)
            else:
                aN = D * A_z / dz
                aS = D * A_z / dz
                rows.append(n); cols.append(idx(i, j + 1)); vals.append(-aN)
                rows.append(n); cols.append(idx(i, j - 1)); vals.append(-aS)
                aP += aN + aS

            rows.append(n); cols.append(n); vals.append(aP)

```

```
    return sp.csr_matrix((vals, (rows, cols)), shape=(N, N)), b

# 9. 径向剖面
def radial_profile_at_mid_z(field, target_cm_local):
    if p.Nz % 2 == 1:
        prof = field[:, p.Nz // 2]
    else:
        j1, j2 = p.Nz // 2 - 1, p.Nz // 2
        prof = 0.5 * (field[:, j1] + field[:, j2])

    f_interp = interp1d(r_cm, prof, kind='linear',
                        bounds_error=False, fill_value=(prof[0], prof[-1]))
    return f_interp(target_cm_local)


target_cm_full = np.arange(0.0, 2.0 + 1e-12, 0.1) # 每 0.1 cm
target_cm_table = np.array([0.0, 0.5, 1.0, 1.5, 2.0]) # 表5

# 10. 从 10800 s 继续算，直到所有位置 C < 0.15
dt_local = 1.0
t = p.t_start

T_inf = T_STABLE
C_inf = C_STABLE
rho_a = rho_a_func(T_inf, C_inf)

record_time = []
record_C = []

next_record_t = 10860.0 # 从 10860 s 开始每 60 s 记录
next_table_t = 6 * 3600.0 # 每 6 h 打印一次表5

print("\n表5 药材烘干过程的水分浓度（每 6 h）")
print("时间/h | " + " | ".join(f"{x:.1f}cm" for x in target_cm_table))
print("-" * 70)

table_rows = []

finished = False
finish_time = None

while t < p.t_max - 1e-9:
    dt_step = min(dt_local, p.t_max - t)
    t += dt_step

    A_T, b_T = assemble_T(T, T_inf, C, rho_a, dt_step)
    T = spla.spsolve(A_T, b_T).reshape((p.Nr, p.Nz), order='F')
```

```
281
282     A_C, b_C = assemble_C(C, C_inf, T, T_inf, dt_step)
283     C = spla.spsolve(A_C, b_C).reshape((p.Nr, p.Nz), order='F')
284     C = np.maximum(C, 1e-6)
285
286     # 判断是否所有位置都小于 0.15
287     if np.all(C < C_LIMIT):
288         finished = True
289         finish_time = t
290         break
291
292     # 每 60 s 记录完整剖面
293     if t >= next_record_t - 1e-9:
294         prof = radial_profile_at_mid_z(C, target_cm_full)
295         record_time.append(int(round(t)))
296         record_C.append(prof)
297         next_record_t += 60.0
298
299     # 每 6 h 打印表5
300     if t >= next_table_t - 1e-9:
301         prof_table = radial_profile_at_mid_z(C, target_cm_table)
302         table_rows.append((t / 3600.0, prof_table))
303         print(f"{t / 3600:>6.1f} | " + " | ".join(f"{v:>8.4f}" for v in prof_table))
304         next_table_t += 6 * 3600.0
305
306 # 如果刚好在循环结束时达到要求，也记录结束时间
307 if finish_time is None and np.all(C < C_LIMIT):
308     finish_time = t
309
310 print("\n" + "=" * 70)
311 if finish_time is not None:
312     print(f"烘干结束时间: {finish_time / 3600.0:.3f} h")
313     print(f"烘干结束时间: {finish_time:.0f} s")
314 else:
315     print(f"在 {p.t_max / 3600.0:.1f} h 内未达到水分浓度全部低于 {C_LIMIT}。")
316
317 # 输出表5完整内容
318 print("\n表5 药材烘干过程的水分浓度")
319 print("时间/h | " + " | ".join(f"{x:.1f}cm" for x in target_cm_table))
320 print("-" * 70)
321 for h, prof in table_rows:
322     print(f"{h:>6.1f} | " + " | ".join(f"{v:>8.4f}" for v in prof))
323 if finish_time is not None:
324     prof_end = radial_profile_at_mid_z(C, target_cm_table)
325     print(f"{finish_time / 3600.0:>6.3f} | " + " | ".join(f"{v:>8.4f}" for v in prof_end))
326
327 # 11. 写入 result3.xlsx
```

```
if len(record_time) > 0:
    columns = ['时间\\到药材中心的距离'] + [f'{x:.1f}' for x in target_cm_full]

    df_C = pd.DataFrame(record_C, columns=columns[1:])
    df_C.insert(0, columns[0], record_time)
    df_C = df_C.round(4)

    out_path = BASE_DIR / 'result3.xlsx'
    with pd.ExcelWriter(out_path, engine='openpyxl') as writer:
        df_C.to_excel(writer, sheet_name='Sheet1', index=False)

    wb = load_workbook(out_path)
    ws = wb['Sheet1']
    for row in ws.iter_rows(min_row=2, min_col=2):
        for cell in row:
            if isinstance(cell.value, (int, float)):
                cell.number_format = '0.0000'
    wb.save(out_path)

    print(f'\n完整结果已写入: {out_path}')
    print(f'记录行数: {len(record_time)}, 时间范围: {record_time[0]} ~ {record_time[-1]} s')
else:
    print('\n没有记录到任何数据。')
```

2.6 问题四方程求解源代码

Listing 6: 问题一耦合方程求解源代码

```
import numpy as np
import pandas as pd
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.interpolate import interp1d
from pathlib import Path
from openpyxl import load_workbook

# 1. 读取数据
BASE_DIR = Path(__file__).resolve().parent
df_env = pd.read_excel(BASE_DIR / '附件1.xlsx')
t_env = df_env['时间'].values.astype(float)
T_env = df_env['温度'].values.astype(float) + 273.15
C_env = df_env['水分浓度'].values.astype(float)
T_env_f = interp1d(t_env, T_env, kind='linear', bounds_error=False, fill_value=(T_env[0],
    T_env[-1]))
C_env_f = interp1d(t_env, C_env, kind='linear', bounds_error=False, fill_value=(C_env[0],
    C_env[-1]))
```

```
17
18  df_shrink = pd.read_excel(BASE_DIR / '附件2.xlsx')
19  t_shrink = df_shrink['时间'].values.astype(float)
20  R_shrink = df_shrink['半径'].values.astype(float)
21  R0_cm = 2.0
22  gamma_data = R_shrink / R0_cm
23  gamma_f = interp1d(t_shrink, gamma_data, kind='linear', bounds_error=False,
        fill_value=(gamma_data[0], gamma_data[-1]))
24
25  # 2. 物理网格参数
26  R0, L0 = 0.02, 0.25
27  dr = 0.0005
28  dz = L0 / 50
29  Nr = int(R0 / dr)
30  Nz = 50
31  N = Nr * Nz
32
33  r = (np.arange(Nr) + 0.5) * dr
34  r_cm = r * 100.0
35  z = -L0/2 + (np.arange(Nz) + 0.5) * dz
36
37  h, hm, Lv = 25.0, 8e-7, 2.26e6
38  T0, C0 = 301.15, 2.55
39  C_LIMIT = 0.15
40  dt = 60.0
41  t_max = 10 * 24 * 3600.0
42
43  table_cm = np.array([0.0, 0.5, 1.0, 1.5, 2.0])
44  record_cm = np.arange(0.0, 2.0 + 1e-12, 0.1)
45
46  # 3. 物性公式
47  def rho_f(C): return 760.0 + 90.0 * C
48  def cp_f(C): return 1850.0 + 2150.0 * C / (C + 1.0)
49  def k_f(C): return 0.12 + 0.20 * C / (C + 1.0)
50  def D_f(C, T): return 4.2e-4 * np.exp(-0.30/np.maximum(C,1e-6)) * np.exp(-3850.0/T)
51
52  A3, A2, A1, A0 = 5.641027e-07, 4.672843e-05, 2.057439e-03, 3.944901e-02
53  T_MID = 308.15
54  def rho_sat(Ts):
55      Ts_clip = np.clip(Ts, 273.15, 323.15)
56      x = Ts_clip - T_MID
57      return np.maximum(A3 * x**3 + A2 * x**2 + A1 * x + A0, 0.0)
58
59  def rho_a_func(Ta, Ca):
60      return Ca * 101325.0 / (287.05 * Ta)
61
62  # 4. 组装矩阵
```

```
def idx(i, j): return i + Nr * j

def assemble_T(T, T_inf, C_field, rho_a, gamma, dot_gamma, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    R_phys = gamma * R0
    i_surf = int(R_phys / dr)
    i_surf = max(0, min(i_surf, Nr - 1))

    for j in range(Nz):
        for i in range(Nr):
            n = idx(i, j)
            if i > i_surf:
                # 已收缩到外部: 值等于当前表面值
                rows.append(n); cols.append(n); vals.append(1.0)
                b[n] = T[i_surf, j]
                continue

            V = r[i] * dr * dz
            r_plus = r[i] + 0.5*dr
            r_minus = r[i] - 0.5*dr if i > 0 else 0.0
            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

            rho = rho_f(C_field[i, j])
            cp = cp_f(C_field[i, j])
            k = k_f(C_field[i, j])

            aP = rho * cp * V / dt_local
            b[n] = aP * T[i, j]

            if i == i_surf:
                # 表面边界条件: -k dT/dr = h(T-Ta) + hm(rho_s - rho_a)Lv
                aW = k * A_rm / dr
                rows.append(n); cols.append(idx(i-1, j)); vals.append(-aW); aP += aW
                evap = hm * (rho_sat(T[i, j]) - rho_a) * Lv
                aP += h
                b[n] += h * T_inf - evap
            else:
                if i < Nr - 1:
                    aE = k * A_rp / dr
                    rows.append(n); cols.append(idx(i+1, j)); vals.append(-aE); aP += aE
                if i > 0:
                    aW = k * A_rm / dr
                    rows.append(n); cols.append(idx(i-1, j)); vals.append(-aW); aP += aW

```

```
            # 轴向端面边界
            if j == 0:
                coeff = k * A_z / dz
                rows.append(n); cols.append(idx(i, j+1)); vals.append(-coeff); aP += coeff
                evap = hm * (rho_sat(T[i, j]) - rho_a) * Lv
                aP += h * A_z / V * dz
                b[n] += h * T_inf * A_z / V * dz - evap * A_z / V * dz
            elif j == Nz - 1:
                coeff = k * A_z / dz
                rows.append(n); cols.append(idx(i, j-1)); vals.append(-coeff); aP += coeff
                evap = hm * (rho_sat(T[i, j]) - rho_a) * Lv
                aP += h * A_z / V * dz
                b[n] += h * T_inf * A_z / V * dz - evap * A_z / V * dz
            else:
                aN = k * A_z / dz
                aS = k * A_z / dz
                rows.append(n); cols.append(idx(i, j+1)); vals.append(-aN)
                rows.append(n); cols.append(idx(i, j-1)); vals.append(-aS)
                aP += aN + aS

            rows.append(n); cols.append(n); vals.append(aP)

    return sp.csr_matrix((vals, (rows, cols)), shape=(N, N)), b

def assemble_C(C, C_inf, T_field, T_inf, rho_a, gamma, dot_gamma, dt_local):
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    R_phys = gamma * R0
    i_surf = int(R_phys / dr)
    i_surf = max(0, min(i_surf, Nr - 1))

    for j in range(Nz):
        for i in range(Nr):
            n = idx(i, j)
            if i > i_surf:
                # 已收缩到外部: 值等于当前表面值
                rows.append(n); cols.append(n); vals.append(1.0)
                b[n] = C[i_surf, j]
                continue

            V = r[i] * dr * dz
            r_plus = r[i] + 0.5*dr
            r_minus = r[i] - 0.5*dr if i > 0 else 0.0
            A_rp = r_plus * dz
            A_rm = r_minus * dz
            A_z = r[i] * dr

```

```
            D = D_f(C[i, j], T_field[i, j])

            aP = V / dt_local
            b[n] = aP * C[i, j]

            if i == i_surf:
                # 表面传质边界: -rho D dC/dr = hm(rho_s - rho_a)
                aW = D * A_rm / dr
                rows.append(n); cols.append(idx(i-1, j)); vals.append(-aW); aP += aW
                aP += hm
                b[n] += hm * (rho_sat(T_field[i, j]) - rho_a)
            else:
                if i < Nr - 1:
                    aE = D * A_rp / dr
                    rows.append(n); cols.append(idx(i+1, j)); vals.append(-aE); aP += aE
                if i > 0:
                    aW = D * A_rm / dr
                    rows.append(n); cols.append(idx(i-1, j)); vals.append(-aW); aP += aW

            if j == 0:
                coeff = D * A_z / dz
                rows.append(n); cols.append(idx(i, j+1)); vals.append(-coeff); aP += coeff
                aP += hm * A_z / V * dz
                b[n] += hm * (rho_sat(T_field[i, j]) - rho_a) * A_z / V * dz
            elif j == Nz - 1:
                coeff = D * A_z / dz
                rows.append(n); cols.append(idx(i, j-1)); vals.append(-coeff); aP += coeff
                aP += hm * A_z / V * dz
                b[n] += hm * (rho_sat(T_field[i, j]) - rho_a) * A_z / V * dz
            else:
                aN = D * A_z / dz
                aS = D * A_z / dz
                rows.append(n); cols.append(idx(i, j+1)); vals.append(-aN)
                rows.append(n); cols.append(idx(i, j-1)); vals.append(-aS)
                aP += aN + aS

            rows.append(n); cols.append(n); vals.append(aP)

    return sp.csr_matrix((vals, (rows, cols)), shape=(N, N)), b

# 5. 求解循环
T = np.full((Nr, Nz), T0)
C = np.full((Nr, Nz), C0)

record_time, record_C, record_surf = [], [], []
table_rows = []
next_table_t = 6 * 3600.0
```

```
t = 0.0

print("\n开始计算（物理坐标，考虑收缩）...")
print("表6 药材烘干过程的水分浓度")
print("时间/h | " + " | ".join(f"{x:.1f}cm" for x in table_cm) + " | 药材表面")
print("-" * 80)

def fmt(v):
    return f"{v:>8.4f}" if not np.isnan(v) else " ---"

finished = False
finish_time = None

while t < t_max - 1e-9:
    gamma = float(gamma_f(t))
    dot_gamma = (float(gamma_f(t+dt)) - float(gamma_f(t))) / dt
    T_inf = float(T_env_f(t))
    C_inf = float(C_env_f(t))
    rho_a = rho_a_func(T_inf, C_inf)

    A_T, b_T = assemble_T(T, T_inf, C, rho_a, gamma, dot_gamma, dt)
    T = spla.spsolve(A_T, b_T).reshape((Nr, Nz), order='F')

    A_C, b_C = assemble_C(C, C_inf, T, T_inf, rho_a, gamma, dot_gamma, dt)
    C = spla.spsolve(A_C, b_C).reshape((Nr, Nz), order='F')
    C = np.maximum(C, 1e-6)

    t += dt

    # 当前物理半径和表面值
    R_phys_cm = gamma * R0_cm
    i_surf = int(R_phys_cm / (dr * 100))
    i_surf = max(0, min(i_surf, Nr - 1))
    C_surf = C[i_surf, Nz // 2]

    # 插值函数（轴向中间剖面）
    C_mid = C[:, Nz // 2]
    f_interp = interp1d(r_cm, C_mid, kind='linear', bounds_error=False, fill_value=(C_mid[0],
        C_mid[-1]))

    # 记录 result4（每 60s）：超出当前半径置为 NaN
    prof = f_interp(record_cm)
    prof[record_cm > R_phys_cm] = np.nan
    record_time.append(int(round(t)))
    record_C.append(prof)
    record_surf.append(C_surf)

```

```
    # 表6（每 6h）
    if t >= next_table_t - 1e-9:
        prof_table = f_interp(table_cm)
        prof_table[table_cm > R_phys_cm] = np.nan
        table_rows.append((t / 3600.0, prof_table, C_surf))
        print(f"{t/3600:>6.1f} | " + " | ".join(fmt(v) for v in prof_table) + f" |
            {C_surf:>8.4f}")
        next_table_t += 6 * 3600.0

    # 判断烘干结束（只判断有效区域）
    if np.all(C[:i_surf+1, :] < C_LIMIT):
        finished = True
        finish_time = t
        break

print("\n" + "=" * 80)
if finish_time is not None:
    print(f"烘干结束时间: {finish_time/3600.0:.3f} h ({finish_time:.0f} s)")
else:
    print(f"在 {t_max/3600.0:.1f} h 内未达到水分浓度全部低于 {C_LIMIT}。")

if finish_time is not None:
    R_phys_cm = float(gamma_f(finish_time)) * R0_cm
    C_mid = C[:, Nz // 2]
    f_interp = interp1d(r_cm, C_mid, kind='linear', bounds_error=False, fill_value=(C_mid[0],
        C_mid[-1]))
    prof_end = f_interp(table_cm)
    prof_end[table_cm > R_phys_cm] = np.nan
    print(f"{finish_time/3600.0:>6.3f} | " + " | ".join(fmt(v) for v in prof_end) + f" |
        {C_surf:>8.4f}")

# 6. 写 result4.xlsx
if len(record_time) > 0:
    columns = ['时间\\到药材中心的距离'] + [f'{x:.1f}' for x in record_cm] + ['药材表面']
    rows_data = []
    for k in range(len(record_time)):
        rows_data.append([record_time[k]] + list(record_C[k]) + [record_surf[k]])

    df_out = pd.DataFrame(rows_data, columns=columns)
    df_out = df_out.round(4)

    df_out = df_out.fillna("/")

    out_path = BASE_DIR / 'result4.xlsx'
    df_out.to_excel(out_path, index=False)

    wb = load_workbook(out_path)
```

```
    ws = wb.active
    for row in ws.iter_rows(min_row=2, min_col=2):
        for cell in row:
            # 只对数值单元格设置格式，跳过 "/"
            if isinstance(cell.value, (int, float)):
                cell.number_format = '0.0000'
    wb.save(out_path)

    print(f"\n完整结果已写入：{out_path}")
    print(f"记录行数：{len(record_time)}，时间范围：{record_time[0]} ~ {record_time[-1]} s")
```

2.7 灵敏度分析源代码

Listing 7: 灵敏度分析源代码

```
from pathlib import Path
import numpy as np
import pandas as pd
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.interpolate import interp1d
import copy
import matplotlib.pyplot as plt

# 中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

BASE_DIR = Path(__file__).resolve().parent

df_env = pd.read_excel(BASE_DIR / '附件1.xlsx')
t_env = df_env['时间'].values.astype(float)
T_env = df_env['温度'].values.astype(float) + 273.15
C_env = df_env['水分浓度'].values.astype(float)

def make_env_interp(kind='linear'):
    fT = interp1d(t_env, T_env, kind=kind, bounds_error=False,
                  fill_value=(T_env[0], T_env[-1]))
    fC = interp1d(t_env, C_env, kind=kind, bounds_error=False,
                  fill_value=(C_env[0], C_env[-1]))
    return fT, fC


# 参数
class P:
    def __init__(self):
```

```
        self.R = 0.02
        self.L = 0.25
        self.dr = 0.001
        self.Nz = 10
        self.rho0, self.rho1 = 650.0, 128.0
        self.cp0, self.cp1 = 1450.0, 2736.0
        self.k0, self.k1 = 0.21, 0.38
        self.D0 = 2.4e-3
        self.aD = 0.45
        self.bD = 3850.0
        self.h = 25.0
        self.hm = 8.0e-7
        self.Lv = 2.26e6
        self.T0 = 301.15
        self.C0 = 2.55
        self.t_end = 3 * 3600.0
        self.dt = 1.0

    def copy(self):
        return copy.deepcopy(self)


def rho_fun(C, p): return p.rho0 + p.rho1 * C
def cp_fun(C, p): return p.cp0 + p.cp1 * C / (C + 1.0)
def k_fun(C, p): return p.k0 + p.k1 * C / (C + 1.0)
def D_fun(C, T, p):
    C_safe = np.maximum(C, 1e-6)
    return p.D0 * np.exp(-p.aD / C_safe) * np.exp(-p.bD / T)

A3, A2, A1, A0 = 5.641027e-07, 4.672843e-05, 2.057439e-03, 3.944901e-02
T_MID = 308.15
def rho_sat(Ts):
    Ts_clip = np.clip(Ts, 273.15, 323.15)
    x = Ts_clip - T_MID
    return np.maximum(A3*x**3 + A2*x**2 + A1*x + A0, 0.0)

Pa, Ra = 101325.0, 287.05
def rho_a_func(Ta, Ca): return Ca * Pa / (Ra * Ta)

def assemble_T(T, T_inf, C_field, rho_a, dt, p, Nr, r, dr, dz, N, idx):
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    for j in range(p.Nz):
        for i in range(Nr):
            n = idx(i, j)
            V = r[i]*dr*dz
            r_plus = r[i] + 0.5*dr
```

```
            r_minus = r[i] - 0.5*dr if i > 0 else 0.0
            A_rp = r_plus*dz; A_rm = r_minus*dz; A_z = r[i]*dr
            rho = rho_fun(C_field[i,j], p)
            cp  = cp_fun(C_field[i,j], p)
            k   = k_fun(C_field[i,j], p)
            aP = rho*cp*V/dt
            b[n] = aP*T[i,j]
            if i == Nr-1:
                aW = k*A_rm/dr; aP += aW
                rows.append(n); cols.append(idx(i-1,j)); vals.append(-aW)
                evap = p.hm*(rho_sat(T[i,j]) - rho_a)*p.Lv
                aP += p.h*A_rp
                b[n] += p.h*A_rp*T_inf - evap*A_rp
            else:
                if i < Nr-1:
                    aE = k*A_rp/dr
                    rows.append(n); cols.append(idx(i+1,j)); vals.append(-aE)
                    aP += aE
                if i > 0:
                    aW = k*A_rm/dr
                    rows.append(n); cols.append(idx(i-1,j)); vals.append(-aW)
                    aP += aW
            if j == 0:
                coeff = k*A_z/dz; aP += coeff
                rows.append(n); cols.append(idx(i,j+1)); vals.append(-coeff)
                evap = p.hm*(rho_sat(T[i,j]) - rho_a)*p.Lv
                aP += p.h*A_z
                b[n] += p.h*A_z*T_inf - evap*A_z
            elif j == p.Nz-1:
                coeff = k*A_z/dz; aP += coeff
                rows.append(n); cols.append(idx(i,j-1)); vals.append(-coeff)
                evap = p.hm*(rho_sat(T[i,j]) - rho_a)*p.Lv
                aP += p.h*A_z
                b[n] += p.h*A_z*T_inf - evap*A_z
            else:
                aN = k*A_z/dz; aS = k*A_z/dz
                rows.append(n); cols.append(idx(i,j+1)); vals.append(-aN)
                rows.append(n); cols.append(idx(i,j-1)); vals.append(-aS)
                aP += aN + aS
            rows.append(n); cols.append(n); vals.append(aP)
    return sp.csr_matrix((vals,(rows,cols)), shape=(N,N)), b

def assemble_C(C, C_inf, T_field, T_inf, dt, p, Nr, r, dr, dz, N, idx):
    rows, cols, vals = [], [], []
    b = np.zeros(N)
    rho_a = rho_a_func(T_inf, C_inf)
    for j in range(p.Nz):
```

```
        for i in range(Nr):
            n = idx(i,j)
            V = r[i]*dr*dz
            D = D_fun(C[i,j], T_field[i,j], p)
            r_plus = r[i] + 0.5*dr
            r_minus = r[i] - 0.5*dr if i > 0 else 0.0
            A_rp = r_plus*dz; A_rm = r_minus*dz; A_z = r[i]*dr
            aP = V/dt
            b[n] = aP*C[i,j]
            if i == Nr-1:
                aW = D*A_rm/dr; aP += aW
                rows.append(n); cols.append(idx(i-1,j)); vals.append(-aW)
                aP += p.hm*A_rp
                b[n] += p.hm*A_rp*(rho_sat(T_field[i,j]) - rho_a)
            else:
                if i < Nr-1:
                    aE = D*A_rp/dr
                    rows.append(n); cols.append(idx(i+1,j)); vals.append(-aE)
                    aP += aE
                if i > 0:
                    aW = D*A_rm/dr
                    rows.append(n); cols.append(idx(i-1,j)); vals.append(-aW)
                    aP += aW
            if j == 0:
                coeff = D*A_z/dz; aP += coeff
                rows.append(n); cols.append(idx(i,j+1)); vals.append(-coeff)
                aP += p.hm*A_z
                b[n] += p.hm*A_z*(rho_sat(T_field[i,j]) - rho_a)
            elif j == p.Nz-1:
                coeff = D*A_z/dz; aP += coeff
                rows.append(n); cols.append(idx(i,j-1)); vals.append(-coeff)
                aP += p.hm*A_z
                b[n] += p.hm*A_z*(rho_sat(T_field[i,j]) - rho_a)
            else:
                aN = D*A_z/dz; aS = D*A_z/dz
                rows.append(n); cols.append(idx(i,j+1)); vals.append(-aN)
                rows.append(n); cols.append(idx(i,j-1)); vals.append(-aS)
                aP += aN + aS
            rows.append(n); cols.append(n); vals.append(aP)
    return sp.csr_matrix((vals,(rows,cols)), shape=(N,N)), b

# 一次求解，返回 3h 末中心/表面值
def run_case(p):
    fT, fC = make_env_interp('linear')
    Nr = int(round(p.R / p.dr))
    dr = p.dr; dz = p.L / p.Nz; N = Nr * p.Nz
    r = (np.arange(Nr) + 0.5) * dr
```

```
    def idx(i, j): return i + Nr * j

    T = np.full((Nr, p.Nz), p.T0, dtype=float)
    C = np.full((Nr, p.Nz), p.C0, dtype=float)

    t = 0.0
    while t < p.t_end - 1e-9:
        dt_step = min(p.dt, p.t_end - t)
        t += dt_step
        T_inf = float(fT(t)); C_inf = float(fC(t))
        rho_a = rho_a_func(T_inf, C_inf)

        A_T, b_T = assemble_T(T, T_inf, C, rho_a, dt_step, p, Nr, r, dr, dz, N, idx)
        T = spla.spsolve(A_T, b_T).reshape((Nr, p.Nz), order='F')

        A_C, b_C = assemble_C(C, C_inf, T, T_inf, dt_step, p, Nr, r, dr, dz, N, idx)
        C = spla.spsolve(A_C, b_C).reshape((Nr, p.Nz), order='F')
        C = np.maximum(C, 1e-6)

    if p.Nz % 2 == 1:
        j_mid = p.Nz // 2
        T_prof = T[:, j_mid]; C_prof = C[:, j_mid]
    else:
        j1, j2 = p.Nz//2 - 1, p.Nz//2
        T_prof = 0.5*(T[:, j1] + T[:, j2])
        C_prof = 0.5*(C[:, j1] + C[:, j2])

    T_center = T_prof[0] - 273.15
    C_center = C_prof[0]
    C_surface = C_prof[-1]
    return T_center, C_center, C_surface

# 4 参数灵敏度
def sensitivity_4params():
    base = P()
    T_c0, C_c0, C_s0 = run_case(base)
    print(f"基准: T_center={T_c0:.4f}°C, C_center={C_c0:.4f}, C_surface={C_s0:.4f}\n")

    # 4 个主控参数
    params = {
        'h':  ('h', 'mul'),
        'hm': ('hm', 'mul'),
        'DO': ('DO', 'mul'),
        'CO': ('CO', 'mul'),
    }
    deltas = [-0.20, -0.10, 0.0, 0.10, 0.20]

```

```
    # 存储：每个参数在每个扰动下的三个指标
    results = {name: {'delta': [], 'T_center': [], 'C_center': [], 'C_surface': []}
               for name in params}

    for name, (attr, _) in params.items():
        for d in deltas:
            p = base.copy()
            setattr(p, attr, getattr(base, attr) * (1 + d))
            T_c, C_c, C_s = run_case(p)
            results[name]['delta'].append(d)
            results[name]['T_center'].append(T_c)
            results[name]['C_center'].append(C_c)
            results[name]['C_surface'].append(C_s)
            print(f"{name:>3s} {d:+.0%}: T_c={T_c:.4f}, C_c={C_c:.4f}, C_s={C_s:.4f}")

    # 存 Excel
    rows = []
    for name in params:
        for i, d in enumerate(results[name]['delta']):
            rows.append({
                '参数': name, '扰动': d,
                'T_center': results[name]['T_center'][i],
                'C_center': results[name]['C_center'][i],
                'C_surface': results[name]['C_surface'][i],
            })
    pd.DataFrame(rows).to_excel(BASE_DIR / 'sensitivity_q2_4params.xlsx', index=False)

    return results, (T_c0, C_c0, C_s0)

# 四子图可视化
def plot_4subplots(results, base_vals):
    T_c0, C_c0, C_s0 = base_vals
    params = list(results.keys())
    colors = {'h': '#1f77b4', 'hm': '#ff7f0e', 'D0': '#2ca02c', 'C0': '#d62728'}
    labels = {'h': 'h（对流换热）', 'hm': 'h_m（对流传质）',
              'D0': 'D0（扩散前置因子）', 'C0': 'C0（初始含水率）'}

    fig, axes = plt.subplots(2, 2, figsize=(13, 10))
    axes = axes.flatten()

    # 子图1: T_center 相对变化
    ax = axes[0]
    for name in params:
        d = np.array(results[name]['delta'])
        y = np.array(results[name]['T_center'])
        rel = (y - T_c0) / T_c0 * 100
        ax.plot(d * 100, rel, 'o-', color=colors[name], label=labels[name], lw=2, ms=6)
```

```
    ax.axhline(0, color='k', ls='--', lw=1, alpha=0.5)
    ax.set_xlabel('参数扰动 (%)')
    ax.set_ylabel('中心温度相对变化 (%)')
    ax.set_title('(a) 3h末中心温度对参数的灵敏度')
    ax.grid(alpha=0.3); ax.legend(fontsize=9)

    # 子图2: C_center 相对变化
    ax = axes[1]
    for name in params:
        d = np.array(results[name]['delta'])
        y = np.array(results[name]['C_center'])
        rel = (y - C_c0) / C_c0 * 100
        ax.plot(d * 100, rel, 's-', color=colors[name], label=labels[name], lw=2, ms=6)
    ax.axhline(0, color='k', ls='--', lw=1, alpha=0.5)
    ax.set_xlabel('参数扰动 (%)')
    ax.set_ylabel('中心水分浓度相对变化 (%)')
    ax.set_title('(b) 3h末中心水分浓度对参数的灵敏度')
    ax.grid(alpha=0.3); ax.legend(fontsize=9)

    # 子图3: C_surface 相对变化
    ax = axes[2]
    for name in params:
        d = np.array(results[name]['delta'])
        y = np.array(results[name]['C_surface'])
        rel = (y - C_s0) / C_s0 * 100
        ax.plot(d * 100, rel, '^-', color=colors[name], label=labels[name], lw=2, ms=6)
    ax.axhline(0, color='k', ls='--', lw=1, alpha=0.5)
    ax.set_xlabel('参数扰动 (%)')
    ax.set_ylabel('表面水分浓度相对变化 (%)')
    ax.set_title('(c) 3h末表面水分浓度对参数的灵敏度')
    ax.grid(alpha=0.3); ax.legend(fontsize=9)

    # 子图4: 相对灵敏度 S 条形图（取 +20% 扰动）
    ax = axes[3]
    S_T, S_Cc, S_Cs = [], [], []
    for name in params:
        d = np.array(results[name]['delta'])
        idx20 = np.argmin(np.abs(d - 0.20))
        T20 = results[name]['T_center'][idx20]
        Cc20 = results[name]['C_center'][idx20]
        Cs20 = results[name]['C_surface'][idx20]
        S_T.append(((T20 - T_c0)/T_c0)/0.20)
        S_Cc.append(((Cc20 - C_c0)/C_c0)/0.20)
        S_Cs.append(((Cs20 - C_s0)/C_s0)/0.20)

    x = np.arange(len(params))
    w = 0.25
```

```
    ax.bar(x - w, S_T, w, label='T_center', color='#1f77b4')
    ax.bar(x,   S_Cc, w, label='C_center', color='#ff7f0e')
    ax.bar(x + w, S_Cs, w, label='C_surface', color='#2ca02c')
    ax.axhline(0, color='k', lw=1)
    ax.set_xticks(x)
    ax.set_xticklabels([labels[n] for n in params], fontsize=9, rotation=15)
    ax.set_ylabel('相对灵敏度 S = (Δy/y)/(Δp/p)')
    ax.set_title('(d) +20%扰动下的相对灵敏度对比')
    ax.grid(alpha=0.3, axis='y'); ax.legend(fontsize=9)

    plt.tight_layout()
    out = BASE_DIR / 'sensitivity_q2_4subplots.png'
    plt.savefig(out, dpi=200, bbox_inches='tight')
    print(f"\n四子图已保存: {out}")
    plt.show()

if __name__ == '__main__':
    results, base_vals = sensitivity_4params()
    plot_4subplots(results, base_vals)
```

题目文件附录 C　Python 程序输出结果

3.1　问题一拟合曲线输出结果

```
C:\Users\86131\anaconda3\envs\DHX\python.exe
    C:\Users\86131\Desktop\26国赛\A题\附件\问题一\拟合.py
温度区间: 20°C 到 50°C
中心温度 T_mid = 308.15 K (即 35°C)
--------------------------------------------------
三次多项式拟合系数 (从高次到低次):
a3 = 5.641027e-07
a2 = 4.672843e-05
a1 = 2.057439e-03
a0 = 3.944901e-02
--------------------------------------------------
最终拟合公式 (x = Ts - 308.15):
rho_fit(x) = 5.641027e-07*x^3 + 4.672843e-05*x^2 + 2.057439e-03*x + 3.944901e-02
--------------------------------------------------
最大相对误差: 0.2494%

进程已结束，退出代码为 0
```

3.2　问题一方程求解结果

```
C:\Users\86131\anaconda3\envs\DHX\python.exe
    C:\Users\86131\Desktop\26国赛\A题\附件\问题一\第一问耦合.py
正在求解...
第一阶段: 0~1 s
第二阶段: 1~1800 s
  step=300, t=301s, T_c=28.27°C, C_c=2.5137, C_s=2.5160
  step=600, t=601s, T_c=28.96°C, C_c=2.4796, C_s=2.4819
  step=900, t=901s, T_c=29.99°C, C_c=2.4459, C_s=2.4483
  step=1200, t=1201s, T_c=31.24°C, C_c=2.4128, C_s=2.4151
  step=1500, t=1501s, T_c=32.64°C, C_c=2.3801, C_s=2.3823
求解完成!

表 1 30 分钟内药材的温度 (°C)
    时间/s |      0 |     0.5 |     1.0 |     1.5 |     2.0
-------------------------------------------------------------
       0 |  28.000 | 28.000 | 28.000 | 28.000 | 28.000
     100 |  28.048 | 28.058 | 28.072 | 28.072 | 28.094
     300 |  28.308 | 28.337 | 28.376 | 28.377 | 28.439
     600 |  29.033 | 29.082 | 29.148 | 29.148 | 29.253
     900 |  30.081 | 30.145 | 30.232 | 30.232 | 30.369
    1200 |  31.348 | 31.426 | 31.529 | 31.530 | 31.694
    1500 |  32.759 | 32.841 | 32.949 | 32.950 | 33.123
    1800 |  34.224 | 34.311 | 34.426 | 34.427 | 34.611

表 2 30 分钟内药材的水分浓度 (kg/kg)
    时间/s |      0 |     0.5 |     1.0 |     1.5 |     2.0
-------------------------------------------------------------
       0 |  2.5500 | 2.5500 | 2.5500 | 2.5500 | 2.5500
     100 |  2.5369 | 2.5370 | 2.5349 | 2.5327 | 2.5081
     300 |  2.5140 | 2.5139 | 2.5120 | 2.5098 | 2.4849
     600 |  2.4799 | 2.4798 | 2.4779 | 2.4758 | 2.4511
     900 |  2.4463 | 2.4462 | 2.4443 | 2.4422 | 2.4179
    1200 |  2.4131 | 2.4130 | 2.4112 | 2.4091 | 2.3851
    1500 |  2.3804 | 2.3803 | 2.3785 | 2.3765 | 2.3528
    1800 |  2.3482 | 2.3481 | 2.3463 | 2.3443 | 2.3210
已写入 result1.xlsx

进程已结束，退出代码为 0
```

3.3 问题二稳态环境求解结果

```
C:\Users\86131\anaconda3\envs\DHX\python.exe
    C:\Users\86131\Desktop\26国赛\A题\附件\问题一\稳态温湿度.py
稳态温度: 50.003 °C
稳态水分浓度: 0.04997
```

```
进程已结束，退出代码为 0
```

3.4 问题二方程求解结果

```
C:\Users\86131\anaconda3\envs\DHX\python.exe
    C:\Users\86131\Desktop\26国赛\A题\附件\问题二\第二问.py

表3 3小时内药材的温度（°C）
时间/h | 0.0cm | 0.5cm | 1.0cm | 1.5cm | 2.0cm
-----------------------------------------------------------
  0.5 |  32.380 |  32.581 |  33.190 |  34.222 |  35.518
  1.0 |  40.824 |  40.998 |  41.507 |  42.318 |  43.273
  1.5 |  46.237 |  46.321 |  46.566 |  46.951 |  47.383
  2.0 |  48.683 |  48.718 |  48.817 |  48.971 |  49.146
  2.5 |  49.577 |  49.587 |  49.614 |  49.655 |  49.722
  3.0 |  49.895 |  49.900 |  49.917 |  49.949 |  49.992

表4 3小时内药材的水分浓度
时间/h | 0.0cm | 0.5cm | 1.0cm | 1.5cm | 2.0cm
-----------------------------------------------------------
  0.5 |  2.5499 |  2.5484 |  2.5217 |  2.3061 |  1.6890
  1.0 |  2.5220 |  2.4885 |  2.3420 |  1.9869 |  1.4782
  1.5 |  2.3697 |  2.3053 |  2.1019 |  1.7512 |  1.3341
  2.0 |  2.1378 |  2.0711 |  1.8742 |  1.5603 |  1.1997
  2.5 |  1.9077 |  1.8475 |  1.6714 |  1.3925 |  1.0693
  3.0 |  1.7033 |  1.6497 |  1.4924 |  1.2415 |  0.9467

计算完成!
已写入: C:\Users\86131\Desktop\26国赛\A题\附件\问题二\result2.xlsx
温度记录行数: 10801
水分浓度记录行数: 10801

进程已结束，退出代码为 0
```

3.5 问题三方程求解结果

```
C:\Users\86131\anaconda3\envs\DHX\python.exe
    C:\Users\86131\Desktop\26国赛\A题\附件\问题三\第三问.py

表5 药材烘干过程的水分浓度（每 6 h）
时间/h | 0.0cm | 0.5cm | 1.0cm | 1.5cm | 2.0cm
-------------------------------------------------------------------
  6.0 |  0.9015 |  0.8689 |  0.7716 |  0.6092 |  0.4012
```

```
 12.0 |  0.3208 |  0.3049 |  0.2567 |  0.1730 |  0.0281
 18.0 |  0.2101 |  0.2015 |  0.1750 |  0.1264 |  0.0281
 24.0 |  0.1726 |  0.1663 |  0.1467 |  0.1097 |  0.0281
 30.0 |  0.1531 |  0.1480 |  0.1318 |  0.1006 |  0.0281

======================================================================
烘干结束时间: 31.313 h
烘干结束时间: 112727 s

表5 药材烘干过程的水分浓度
时间/h | 0.0cm | 0.5cm | 1.0cm | 1.5cm | 2.0cm
----------------------------------------------------------------------
  6.0 |  0.9015 |  0.8689 |  0.7716 |  0.6092 |  0.4012
 12.0 |  0.3208 |  0.3049 |  0.2567 |  0.1730 |  0.0281
 18.0 |  0.2101 |  0.2015 |  0.1750 |  0.1264 |  0.0281
 24.0 |  0.1726 |  0.1663 |  0.1467 |  0.1097 |  0.0281
 30.0 |  0.1531 |  0.1480 |  0.1318 |  0.1006 |  0.0281
31.313 |  0.1500 |  0.1450 |  0.1293 |  0.0991 |  0.0281

完整结果已写入: C:\Users\86131\Desktop\26国赛\A题\附件\问题三\result3.xlsx
记录行数: 1698, 时间范围: 10860 ~ 112680 s

进程已结束, 退出代码为 0
```

3.6 问题四方程求解结果

```
C:\Users\86131\anaconda3\envs\DHX\python.exe
    C:\Users\86131\Desktop\26国赛\A题\附件\问题四\第四问.py

开始计算（物理坐标, 考虑收缩）...
表6 药材烘干过程的水分浓度
时间/h | 0.0cm | 0.5cm | 1.0cm | 1.5cm | 2.0cm | 药材表面
------------------------------------------------------------------------------
  6.0 |  1.5813 |  1.3634 |  0.7361 |    --- |    --- |  0.0283
 12.0 |  0.5288 |  0.4451 |  0.2149 |    --- |    --- |  0.0283
 18.0 |  0.2742 |  0.2358 |  0.1214 |    --- |    --- |  0.0283
 24.0 |  0.1944 |  0.1707 |  0.0963 |    --- |    --- |  0.0283
 30.0 |  0.1588 |  0.1413 |  0.0843 |    --- |    --- |  0.0283

==============================================================================
烘干结束时间: 32.283 h (116220 s)
32.283 |  0.1500 |  0.1340 |  0.0813 |    --- |    --- |  0.0283

完整结果已写入: C:\Users\86131\Desktop\26国赛\A题\附件\问题四\result4.xlsx
记录行数: 1937, 时间范围: 60 ~ 116220 s
```

```
19
20 进程已结束，退出代码为 0
```

3.7 灵敏度分析结果

```
1  C:\Users\86131\anaconda3\envs\DHX\python.exe
       C:\Users\86131\Desktop\26国赛\A题\附件\问题二\灵敏度分析.py
2  基准: T_center=49.8947°C, C_center=1.7033, C_surface=0.9467
3
4    h -20%: T_c=49.8033, C_c=1.7112, C_s=0.9467
5    h -10%: T_c=49.8591, C_c=1.7067, C_s=0.9468
6    h +0%: T_c=49.8947, C_c=1.7033, C_s=0.9467
7    h +10%: T_c=49.9183, C_c=1.7006, C_s=0.9466
8    h +20%: T_c=49.9345, C_c=1.6984, C_s=0.9464
9   hm -20%: T_c=49.8776, C_c=1.8144, C_s=1.1396
10  hm -10%: T_c=49.8866, C_c=1.7565, C_s=1.0384
11  hm +0%: T_c=49.8947, C_c=1.7033, C_s=0.9467
12  hm +10%: T_c=49.9021, C_c=1.6544, C_s=0.8632
13  hm +20%: T_c=49.9088, C_c=1.6093, C_s=0.7868
14  D0 -20%: T_c=49.8953, C_c=1.8291, C_s=0.9017
15  D0 -10%: T_c=49.8950, C_c=1.7609, C_s=0.9265
16  D0 +0%: T_c=49.8947, C_c=1.7033, C_s=0.9467
17  D0 +10%: T_c=49.8946, C_c=1.6544, C_s=0.9635
18  D0 +20%: T_c=49.8944, C_c=1.6125, C_s=0.9774
19  C0 -20%: T_c=49.9261, C_c=1.3787, C_s=0.7304
20  C0 -10%: T_c=49.9113, C_c=1.5407, C_s=0.8389
21  C0 +0%: T_c=49.8947, C_c=1.7033, C_s=0.9467
22  C0 +10%: T_c=49.8764, C_c=1.8664, C_s=1.0541
23  C0 +20%: T_c=49.8564, C_c=2.0300, C_s=1.1611
24
25 四子图已保存: C:\Users\86131\Desktop\26国赛\A题\附件\问题二\sensitivity_q2_4subplots.pdf
26
27 进程已结束，退出代码为 0
```